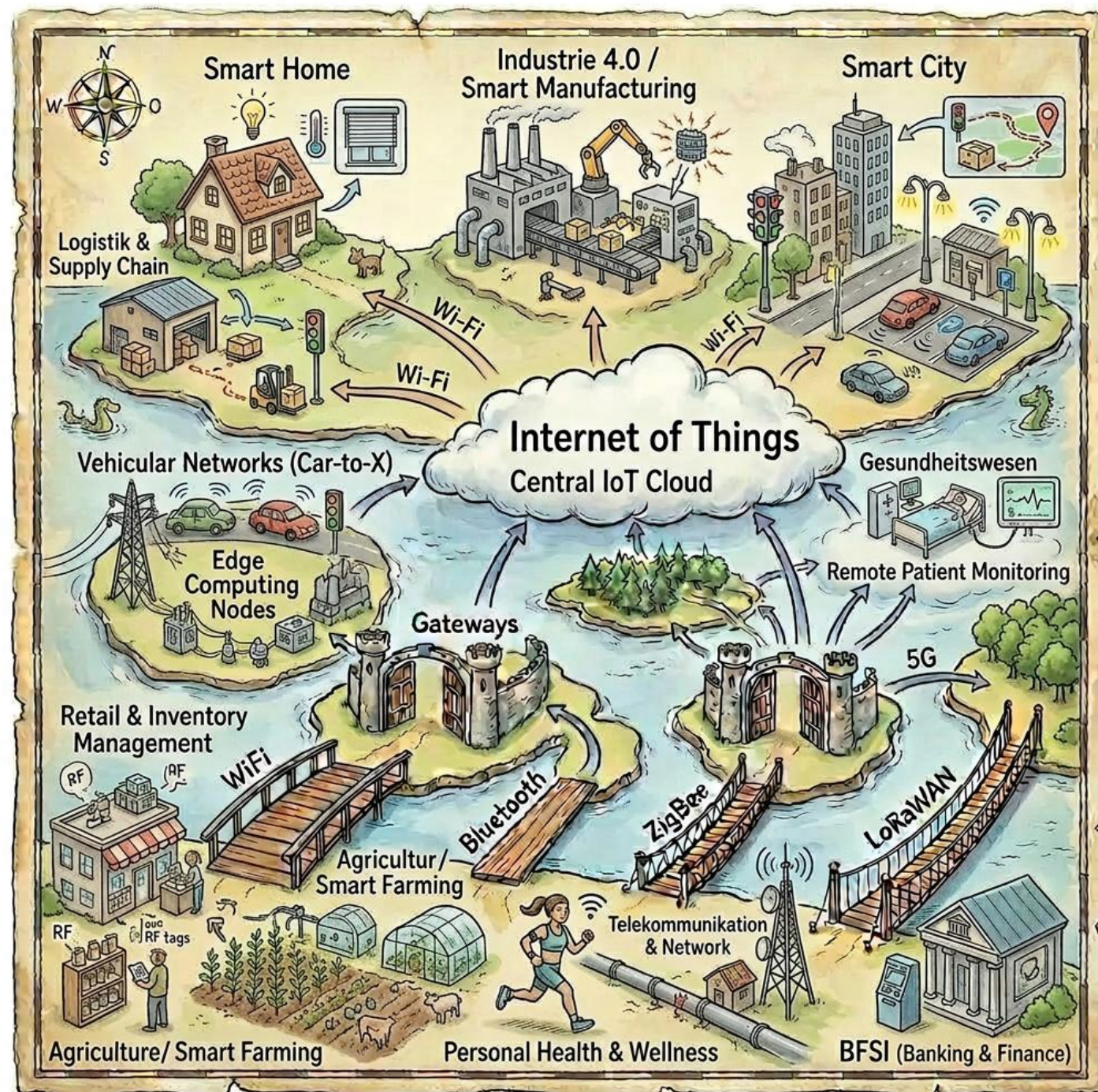
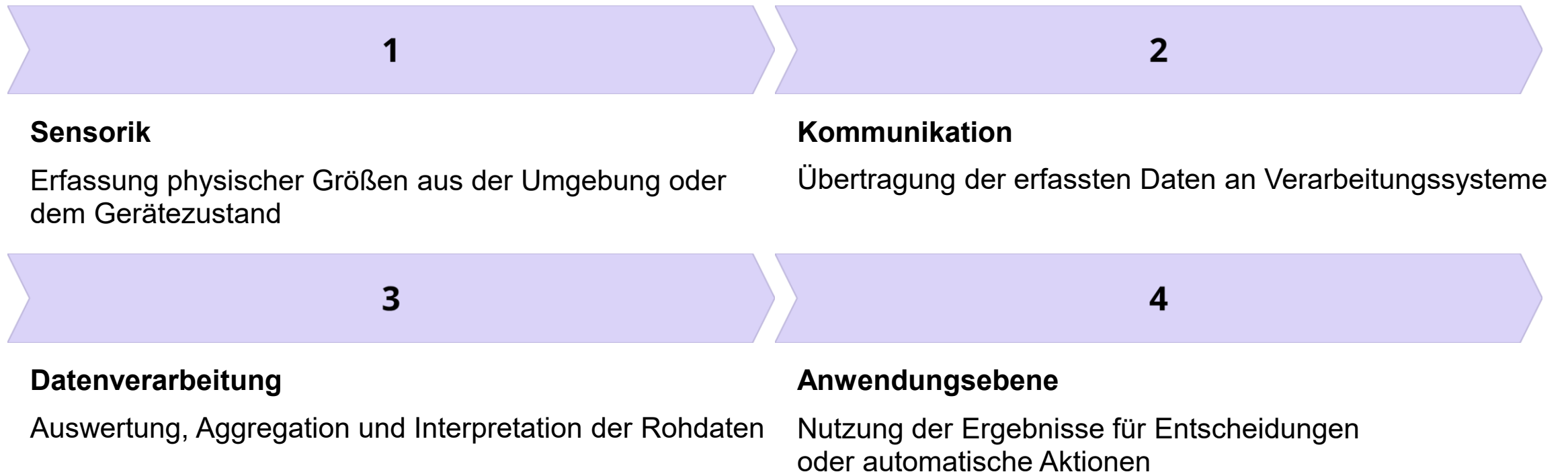


IoT Use Cases und Beispiele



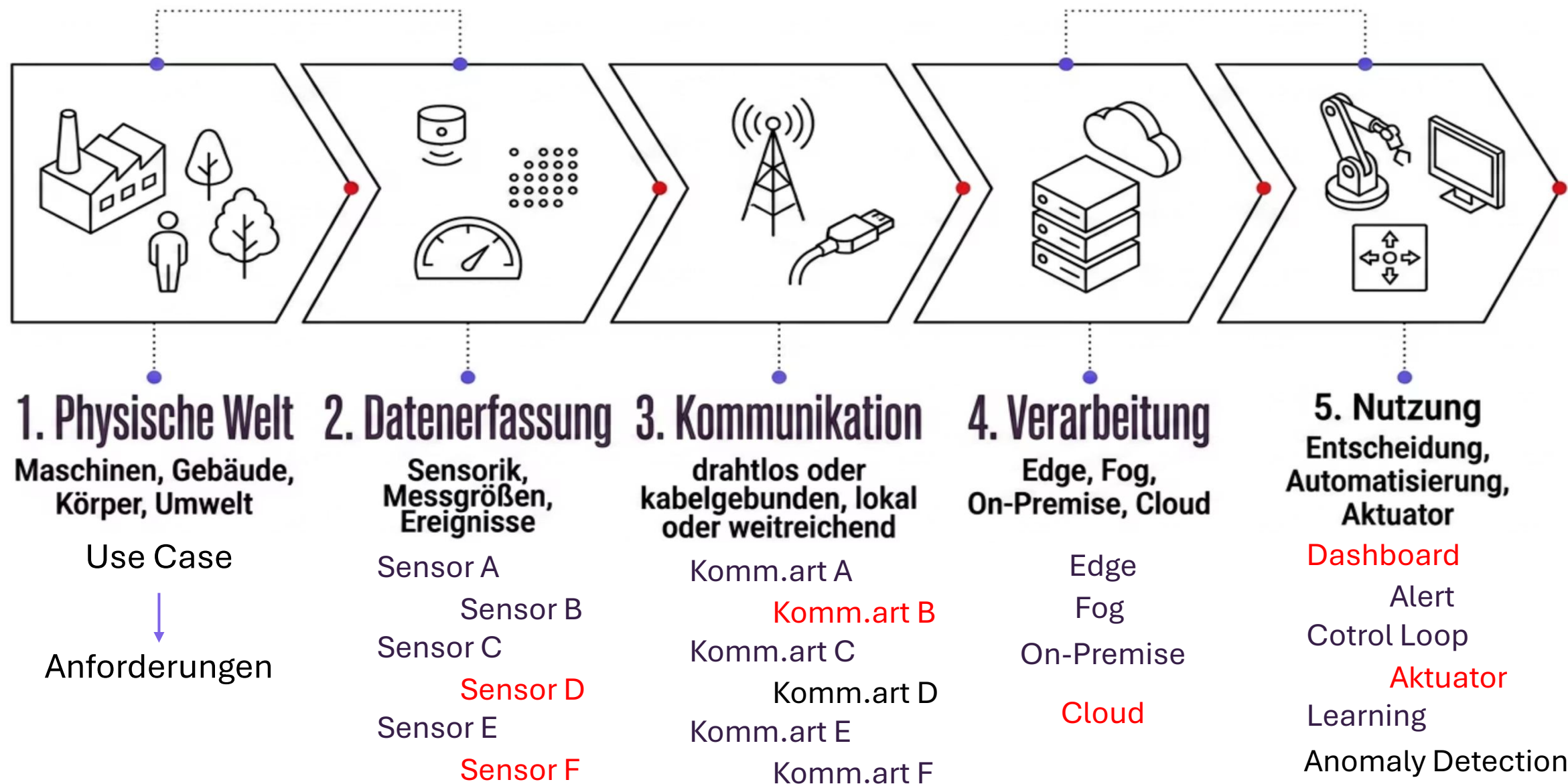
Was ist das Internet of Things?

Das **Internet of Things (IoT)** bezeichnet vernetzte physische Objekte, die Daten aus ihrer Umgebung oder ihrem eigenen Zustand erfassen, übertragen, verarbeiten oder in Aktionen umsetzen. Ein IoT-System verbindet typischerweise vier Ebenen miteinander:



Im Unterschied zur klassischen IT steht nicht nur Informationsverarbeitung im Zentrum, sondern die **Kopplung an die physische Welt**: Messen, Kommunizieren, Reagieren.

Grundidee



Ein Top-down-Ansatz

Zuerst der **Anwendungskontext**, daraus leiten wir die technische Anforderungen ab.
Die Vielfalt der IoT-Technologien ist eine direkte Folge der Vielfalt der Use Cases – nicht umgekehrt.

Anwendung zuerst

Ein Smart Meter, ein Wearable und ein Fahrzeugkommunikationssystem stellen grundlegend unterschiedliche Anforderungen: Reichweite, Latenz, Energiebedarf und Zuverlässigkeit variieren erheblich je nach Use Case.

Technologie als Antwort

Funkstandards, Protokolle und Architekturen sind keine willkürlichen Wahlmöglichkeiten, sondern ingenieurstechnische Antworten auf konkrete Anforderungen aus den Anwendungsdomänen.

Grundlagen als Templates

Die verwendeten Technologien haben oft gleiche Zielsetzungen, sind nur unterschiedliche Ausgestaltungen. Wir lernen die Grundlagen kennen, so dass auch neue Technologien in das Bild einsortiert werden können.



Überblick: Eine Auswahl an IoT-Kategorien

Die folgende Liste zeigt 15 Use-Case-Kategorien, geordnet nach geschätzter Verbreitung. **Consumer-Anwendungen** wie Smart Home dominieren bei der Geräteanzahl, während **industrielle Use Cases** wie Industrie 4.0 höhere Investitionen und Return on Investment (ROI) pro Projekt erzielen.

Beispiele

1. Smart Home / Facility Management
2. Industrie 4.0 / Smart Manufacturing
3. Smart City / Umwelt / Agriculture
4. Vehicular Networks (V2X)
5. Remote Patient Monitoring / Personal Health
6. Smart Grid / Öl & Gas / Telekommunikation
7. Logistik & Supply Chain / Retail

Industrie 4.0: Rezipienten, Aktuatoren, Erfolg und Praxisbeispiel

Rezipienten der Daten

Instandhaltungsteams, Produktionsplanung, Leitstände und das Management.

Daten fließen in Manufacturing Execution Systems (MES) und übergeordnete ERP-Systeme.

Aktuatoren

Alarmer, automatisch ausgelöste Wartungsaufträge, Anpassungen von Maschinenparametern und in fortgeschrittenen Systemen vollautomatische Prozessanpassungen ohne menschlichen Eingriff.

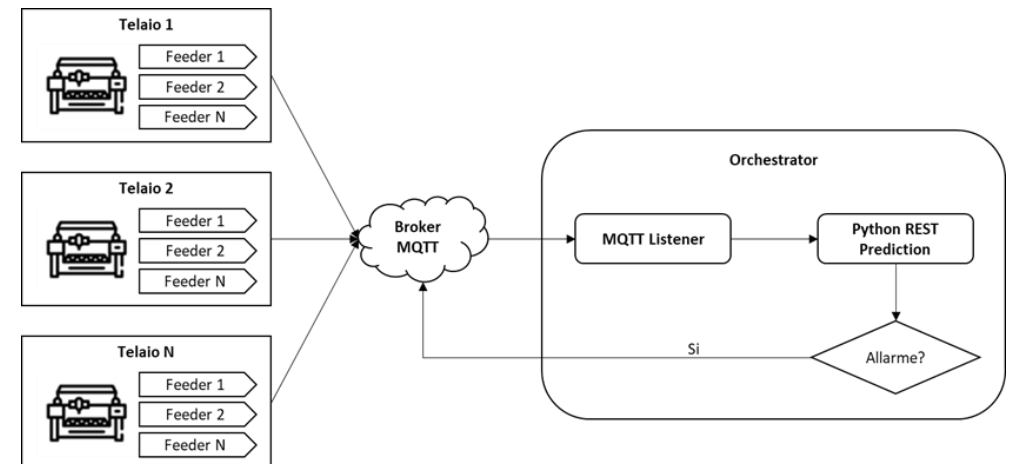
Erfolgskriterien

Ungeplante Stillstände sinken messbar, Wartung erfolgt gezielter und die Produktion wird insgesamt robuster und effizienter.

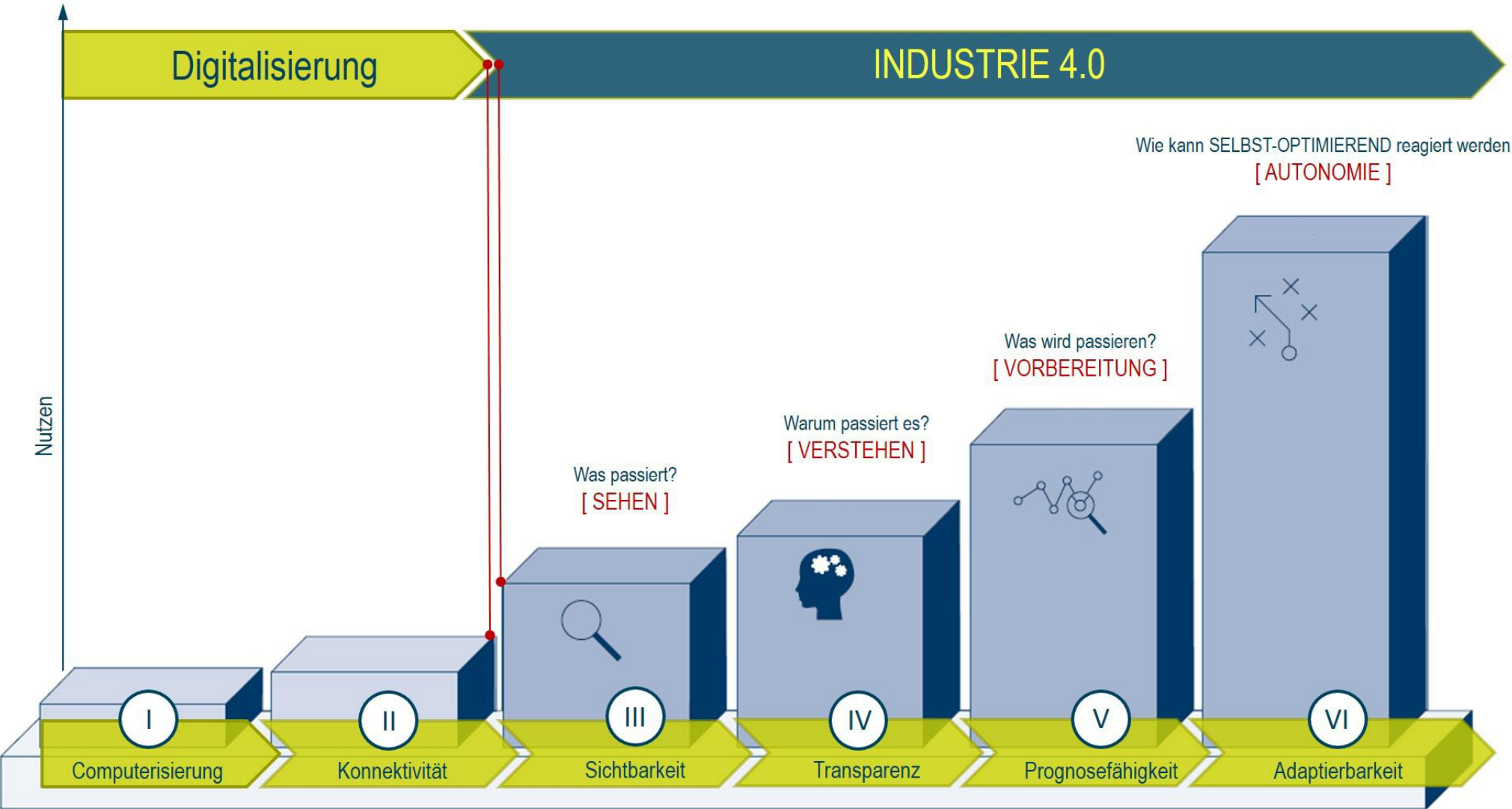
Praxisbeispiel

Revelis dokumentiert eine Predictive-Maintenance-Fallstudie. Durch Vibrationssensorik an Maschinen konnte ein drohender Lagerausfall frühzeitig erkannt und ein ungeplanter Produktionsstillstand verhindert werden.

revelis.eu – Predictive Maintenance Case Study



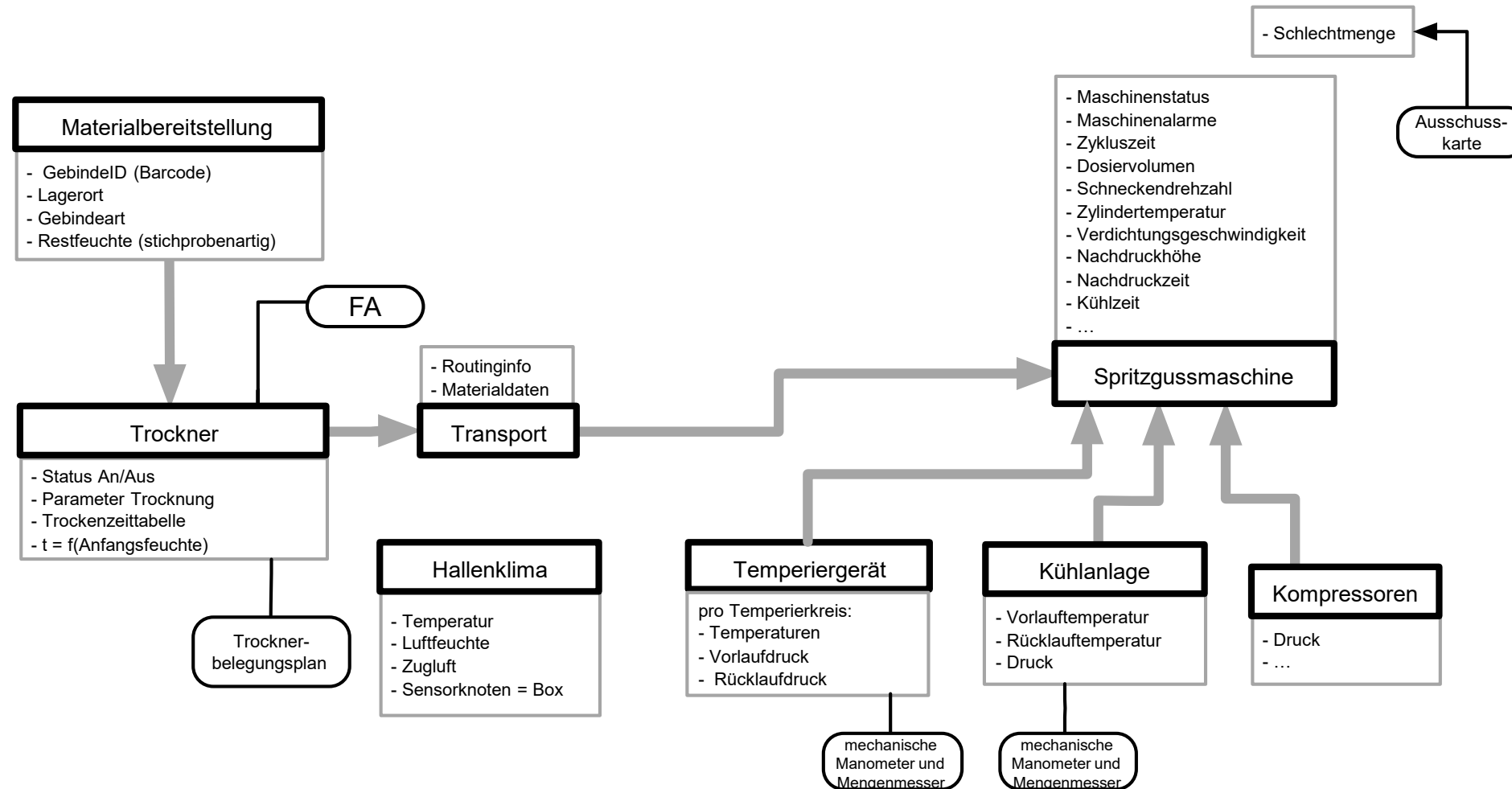
Maturity-Index des FIR



Stufen des INDUSTRIE 4.0-Entwicklungspfad (in Anlehnung an eine Grafik des FIR e.V. / RWTH AACHEN)

Quelle: FIR

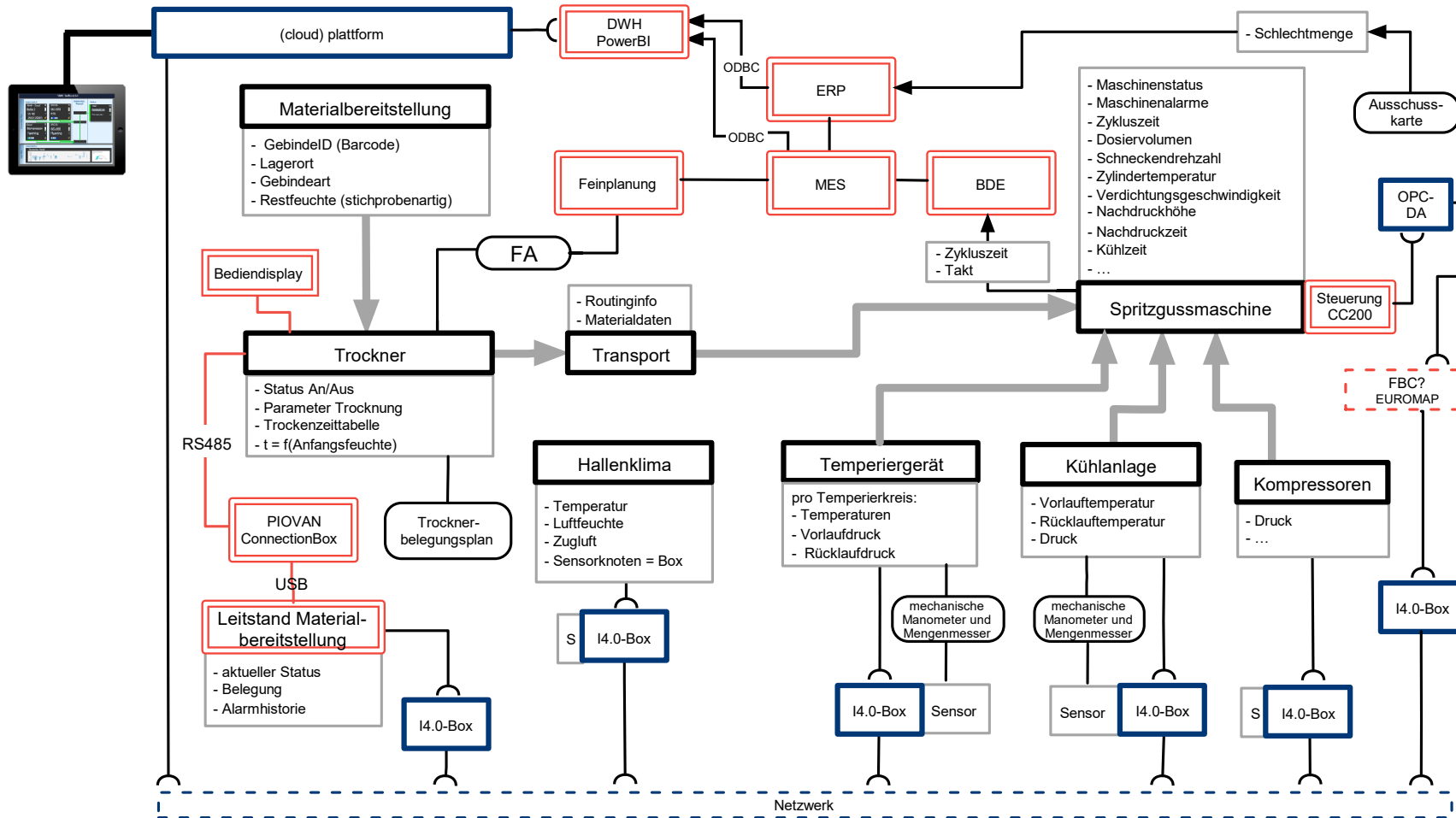
Anwendungsfall: Ganzheitliches Monitoring im Spritzguss Prozess



FA: Fertigungsauftrag

Retrofit mit der I4.0-Box

Beispiel Ganzheitliches Monitoring im Spritzguss



FA: Fertigungsauftrag
 ERP: Enterprise Resource Planing System
 MES: Manufacturing Execution System
 BDE: Betriebsdatenerfassung
 ODBC: Open Database Connectivity
 DWH: Data Warehouse

Smart City: Definition und Abgrenzung

Smart City umfasst vernetzte Systeme zur Erfassung, Analyse und Steuerung urbaner Infrastruktur und öffentlicher Dienste. Ziel ist eine intelligentere, effizientere und nachhaltigere Stadtentwicklung auf Basis von Echtzeit-Daten.

Typische Bereiche

Verkehrsmanagement, Parkraumüberwachung, adaptive Straßenbeleuchtung, Umweltmonitoring, Abfallentsorgung (Füllstandsüberwachung von Containern) und öffentliche Sicherheit.

Abgrenzung

Die Kategorie endet dort, wo einzelne private IoT-Systeme ohne kommunalen oder städtischen Zusammenhang betrachtet werden. Entscheidend ist der öffentliche oder infrastrukturelle Charakter des Systems.

Smart Cities

Trend¹:

- 2014: 54%, or 3.3bn people lived in Urban areas
 - 2030: 66%, or 5bn people, will live in Urban areas
- How to build and manage cities improving the lives of billions?
- How to use resources in cities efficiently?

Smart Cities²:

*“A **smart sustainable city** is an innovative city that **uses information and communication technologies (ICTs)** and other means **to improve quality of life, competitiveness and efficiency** of urban operation and services, while ensuring that it meets the needs of present and future generations **with respect to economic, social and environmental aspects**”*

¹ United Nations Population Fund

² ITU Focus Group on Smart Sustainable Cities

Smart City: Daten, Ziele und Kommunikationsbedarf

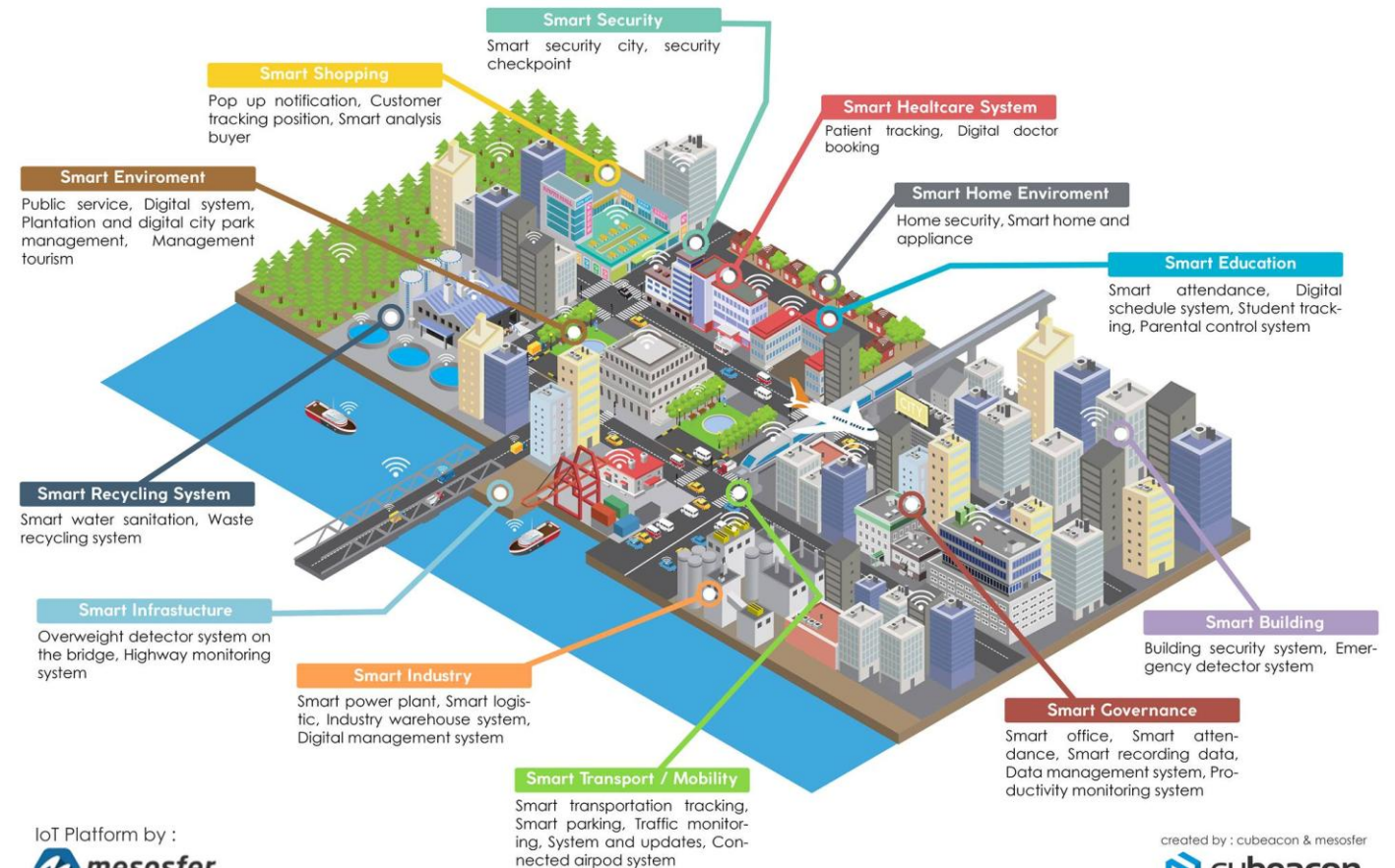
Erhobene Daten

Verkehrsflussdaten, Parkplatzbelegung, Luftqualitätswerte, Füllstände von Abfallbehältern, Wasserstände, Energieverbrauch öffentlicher Infrastruktur und Lärmbelastung.

Ziele der Auswertung

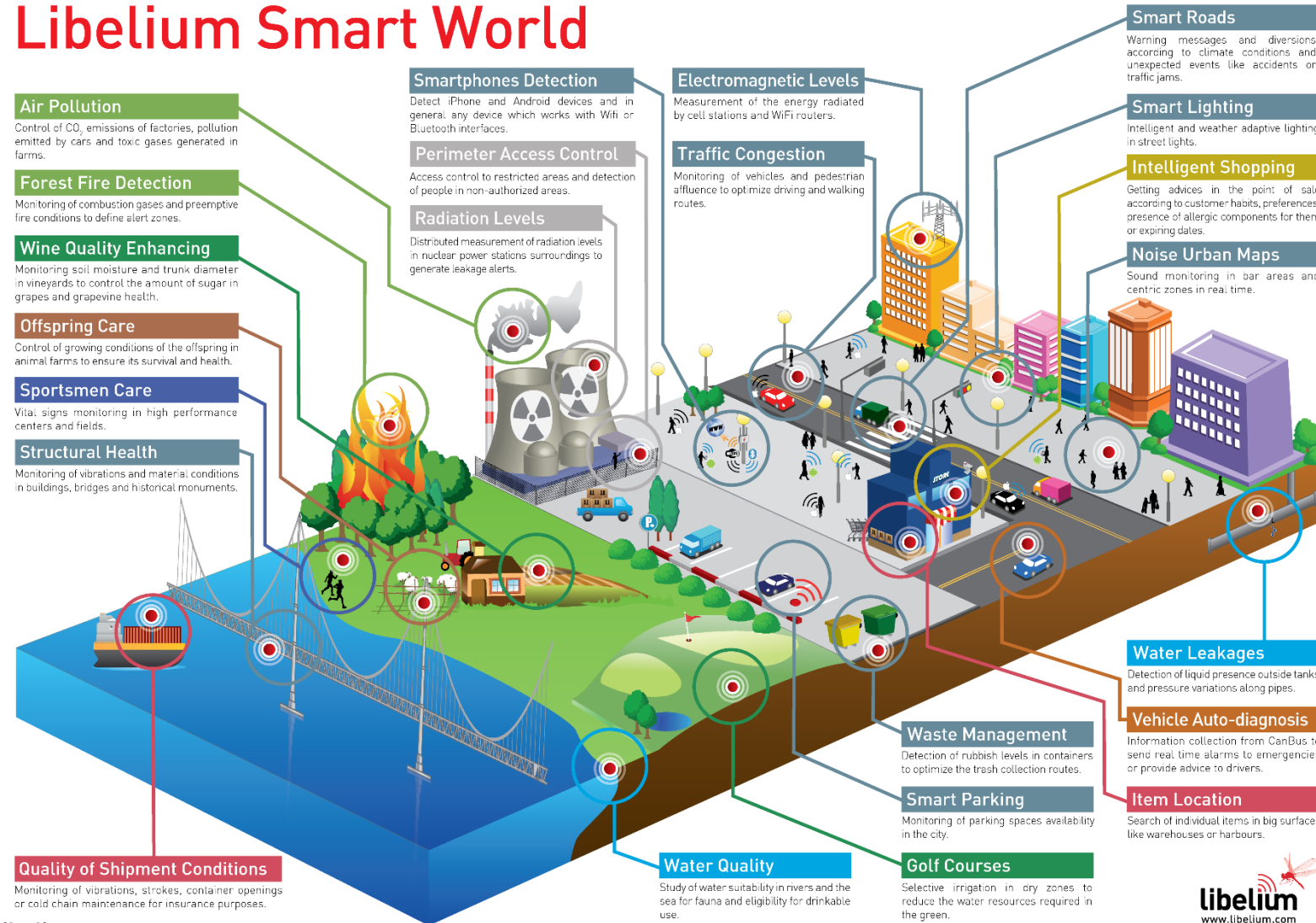
Effizientere Stadtsteuerung, verbesserte Nachhaltigkeit, geringere Betriebskosten für Kommunen und spürbar verbesserte Lebensqualität für Bürgerinnen und Bürger.

SMART DIGITAL LIFE



Smart City Use Case

Libelium Smart World



Source: Libelium.com

Umwelt und Nachhaltigkeit: Definition und Abgrenzung

Umweltmonitoring mit IoT umfasst Systeme zur Beobachtung von Umweltzuständen in natürlichen, ländlichen oder städtischen Umgebungen. Ziel ist die kontinuierliche, flächendeckende und automatisierte Erfassung von Umweltparametern.

Typische Anwendungen

Luftqualitätsmonitoring in Städten (Feinstaub, Stickstoffdioxid), Pegelüberwachung an Gewässern, Hochwasserfrühwarnsysteme, Wetterstationen, Lärmerfassung in Wohngebieten und Bodensensorik in Schutzgebieten.

Verhältnis zur Smart City

Umweltmonitoring kann **Teil einer Smart City** sein (z. B. städtische Luftqualitätsnetze), ist aber auch als eigenständiger Use Case relevant – etwa in Naturschutzgebieten, Küstenregionen oder Industriestandorten ohne städtischen Kontext.



Umwelt und Nachhaltigkeit: Rezipienten, Aktuatoren, Erfolg und Praxisbeispiel

Rezipienten der Daten

Umweltbehörden, Forschungsinstitute, Kommunen und Katastrophenschutzbehörden sowie teils die Öffentlichkeit über Open-Data-Portale oder Warn-Apps.

Aktuatoren (überwiegend indirekt)

Warnmeldungen an Bevölkerung und Behörden, Sperrungen von Flächen, Alarmierung von Einsatzkräften (z. B. Feuerwehr, THW) und operative Maßnahmen wie Pumpenbetrieb oder Straßensperrungen.

Erfolgskriterien

Daten sind verlässlich, räumlich ausreichend dicht verteilt und zeitnah verfügbar. Sie werden tatsächlich für Entscheidungen genutzt und haben nachweislichen Einfluss auf politische oder operative Prozesse.

Praxisbeispiel

LORIoT beschreibt IoT-Umweltanwendungen als Wachstumsfeld für 2025. Städte wie Zürich und Amsterdam betreiben LoRaWAN-basierte Umwelt-Sensornetze mit hunderten Knoten zur kontinuierlichen Luftqualitätsmessung.

loriot.io – IoT Trends 2025

Regularien und gesetzliche Verpflichtungen

Bundes-Klimaschutzgesetz (KSG) - § 3 Nationale Klimaschutzziele

(1) Die Treibhausgasemissionen werden im Vergleich zum Jahr 1990 schrittweise wie folgt gemindert:

1. bis zum Jahr 2030 um mindestens 65 Prozent,
2. bis zum Jahr 2040 um mindestens 88 Prozent.

(2) **Bis zum Jahr 2045 werden die Treibhausgasemissionen so weit gemindert, dass Netto-Treibhausgasneutralität erreicht wird.**

Nach dem Jahr 2050 sollen negative Treibhausgasemissionen erreicht werden.

Netto-Treibhausgasneutralität bedeutet, dass ein **Gleichgewicht zwischen** den durch Menschen verursachten Treibhausgasemissionen und deren **Abbau** durch Senken (wie Wälder oder technische Speicher) herrscht

Gesetz zur Steigerung der Energieeffizienz in Deutschland¹ (Energieeffizienzgesetz - EnEfG) - § 4 Energieeffizienzziele

(1) Ziel dieses Gesetzes ist es,

1. den **Endenergieverbrauch** Deutschlands im Vergleich zum Jahr 2008 **bis zum Jahr 2030 um mindestens 26,5 Prozent** auf einen Endenergieverbrauch von 1 867 Terawattstunden zu senken,
2. den **Primärenergieverbrauch** Deutschlands im Vergleich zum Jahr 2008 **bis zum Jahr 2030 um mindestens 39,3 Prozent** auf einen Primärenergieverbrauch von 2 252 Terawattstunden zu senken.

(2) Für den Zeitraum nach 2030 strebt die Bundesregierung an, den **Endenergieverbrauch** Deutschlands im Vergleich zum Jahr 2008 **bis zum Jahr 2045 um 45 Prozent zu senken**. Die Energieeinspargrößen nach Satz 1 wird die Bundesregierung im Jahr 2027 überprüfen und dem Deutschen Bundestag einen Bericht zur Fortschreibung der Energieeffizienzziele für den Zeitraum nach 2030 vorlegen.

Regularien und gesetzliche Verpflichtungen

Aktueller Kohlestoffkreislauf:

- Emittenten: Natur und menschlich (anthropogen)

Emission:

- **Ca 830 Gigatonnen CO₂ jährlich** in die Atmosphäre
- Ca 790 GtCO₂ natürlich, 40 GtCO₂ (ca 5%) anthropogen
- Achtung oft wird der na

Senken bauen ca. 810 GtCO₂ pro Jahr wieder ab! (98%)

- Ozeane (Partialdruck pusht CO₂ ins Wasser)
- Vegtation (überwiegend auf Nordhemisphäre)

Verbleiben 20 GtCO₂ zusätzlich in der Atmosphäre

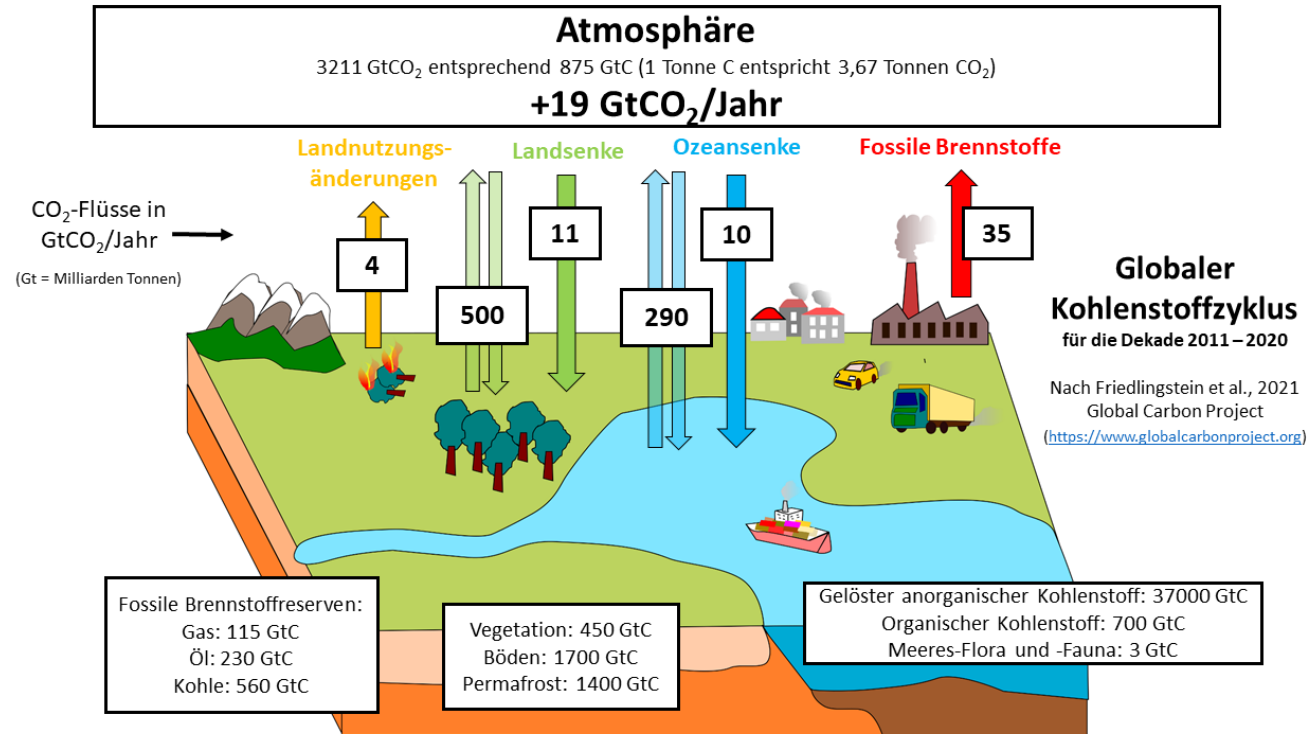
- Akkumulation über viele Jahre → Treibhauseffekt

Lösung:

- **Anthropogene CO₂ Emmission von 40 auf 20 GtCO₂ senken**
- Oder Senkenleistung um 3% steigern.

* <https://www.ess.uci.edu/~reeburgh/figures.html>

<https://www.pik-potsdam.de/de/aktuelles/nachrichten/archiv/2009/co2-und-temperatur>



Global Carbon Budget 2021 - <https://essd.copernicus.org/articles/14/1917/2022/>

Die Natur baut 98% der jährlichen CO₂ Emmission ab! Häufige falsche Darstellung:

Natur baut nur 50% des anthropogenen CO₂ ab!
(Man müsste die Senkenleistung verdoppeln - falsch)

Regularien und gesetzliche Verpflichtungen

Vorgabe:

- **Anthropogene CO₂ Emission von 40 auf 20 GtCO₂ senken (-50%!)**
- Oft verwechselt: netto-null (-50% CO₂-Ausstoß) und Null-Emission 100%

Beitrag von IoT:

- Vorgaben reichen in fast alle Industrien.
- Erfassung des Ausstoßes, Energieeffizienzsteigerung, optimierte Prozesse

Minderung der deutschen Treibhausgasemissionen gegenüber 1990:

1990	1.242	0,0 %
2000	1.037	-16,5 %
2010	936	-24,7 %
2021	762	-38,7 %
2030	435	-65,0 %
2040	149	-88,0 %
2045	Netto-Treibhausgasneutralität	

Millionen Tonnen
CO₂-Äquivalente

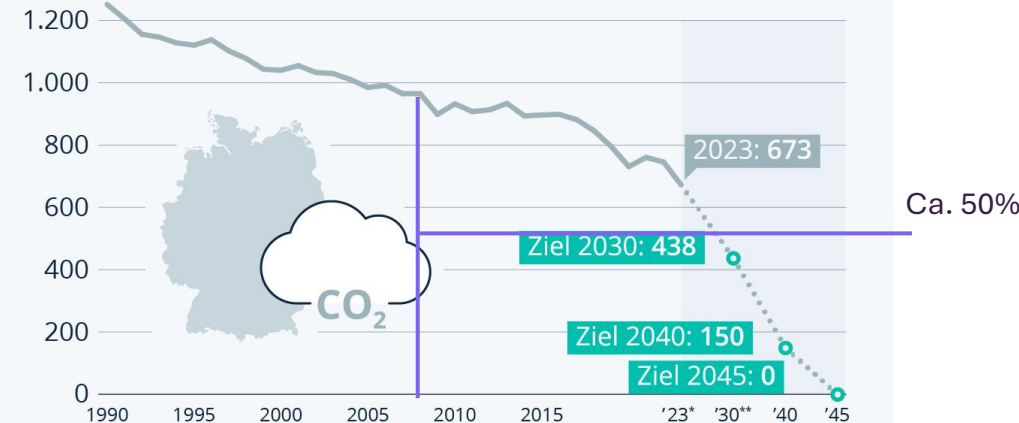
Deutsche Treibhausgasemissionen nach Sektoren:

Stand 2021	247	181	148	115	61	8
Ziel 2030*	108	118	85	67	56	4

⚡ Energiewirtschaft 🏭 Industrie 🚚 Verkehr
🏠 Gebäude 🌾 Landwirtschaft ♻️ Abfallwirtschaft und Sonstige

*Maximal zulässige Jahresemissionsmengen nach Klimaschutzgesetz

Entwicklung der Treibhausgasemissionen in Deutschland
(in Mio. Tonnen CO₂-Äquivalent)



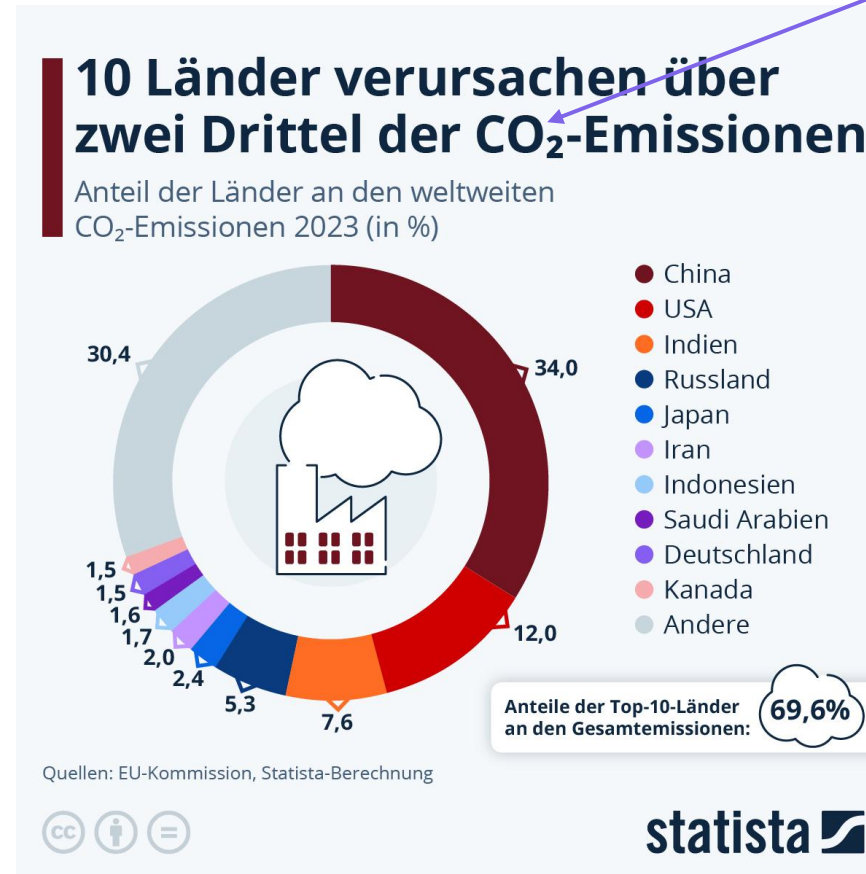
* vorläufige Berechnung ** Ziele gemäß Bundes-Klimaschutzgesetz
Quellen: Agora Energiewende, Bundesregierung, UBA



statista

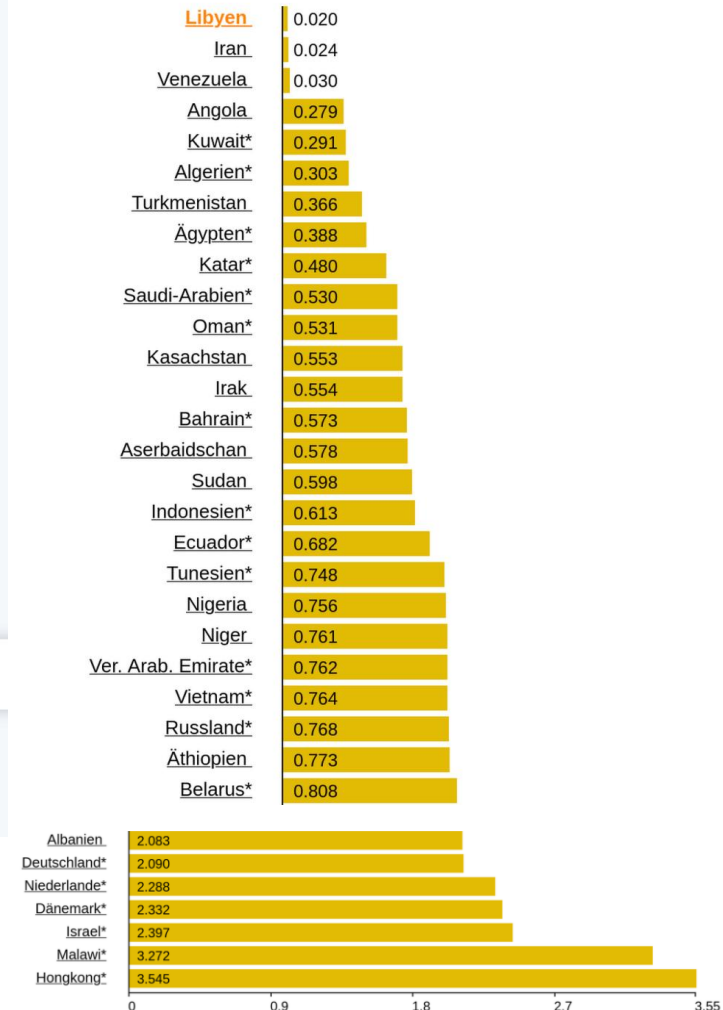
Regularien und gesetzliche Verpflichtungen

Mit IoT lässt sich viel erreichen.
- Perspektivisch kann Deutschland
0,075% CO₂ Emission einsparen.
(1,5% der anthropogenen CO₂ Emission
von 5% der Gesamtemissionen)



anthropogenen

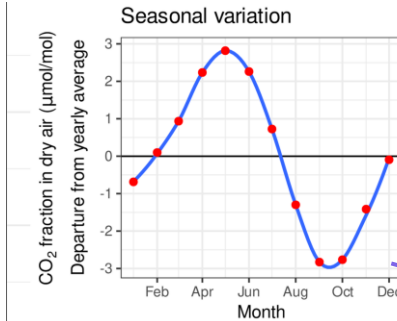
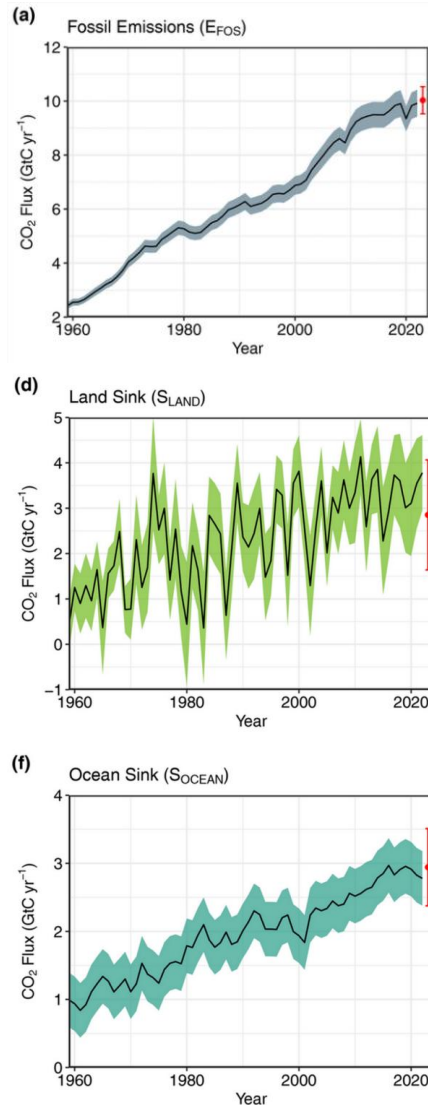
Benzinpreise, 27-Apr-2026
(liter, Euro)



<https://de.statista.com/infografik/23383/anteil-der-laender-an-co2-emissionen/>

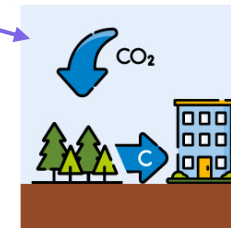
https://de.globalpetrolprices.com/Libya/gasoline_prices/

Möglichkeiten zur Steigerung der CO₂-Senken (um 3%+)



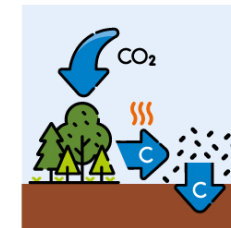
Kohlenstoffsinken in der Land- und Forstwirtschaft

Das natürliche Pflanzenwachstum bietet mehrere Möglichkeiten CO₂ aus der Atmosphäre aufzunehmen und langfristig zu binden.



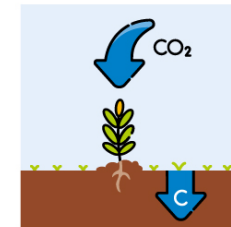
Aufforstung

Baumwachstum entzieht der Atmosphäre CO₂. Holz als Baustoff kann Kohlenstoff langfristig binden.



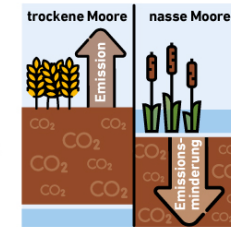
Pflanzenkohle

Pflanzenkohle (auch Biokohle) wird nur langsam von Mikroorganismen zersetzt und bindet so den Kohlenstoff langfristig im Boden.



Aufbau organischer Bodensubstanz

Gründüngung, Untersaaten, Beweidung von Dauergrünland und Agroforstsysteme erhöhen den Kohlenstoffgehalt im Boden.

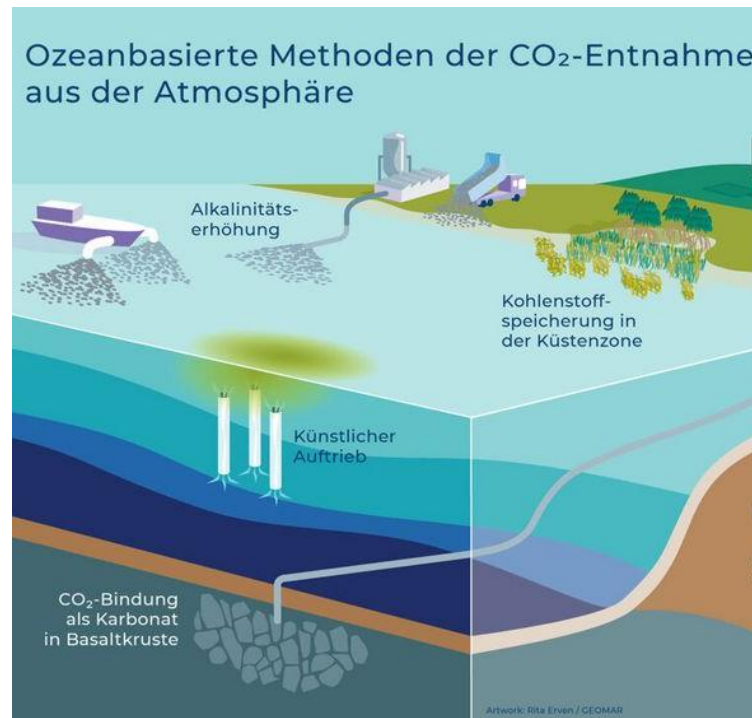


Wiedervernässung

Die Sauerstoffarmut in Feuchtgebieten sorgt dafür, dass abgestorbene Pflanzenteile nicht verrotten. Dadurch kann der darin enthaltene Kohlenstoff nicht in die Atmosphäre entweichen.

Quelle: eigene Darstellung; Stand: 5/2021

© 2021 Agentur für Erneuerbare Energien e.V.



Personal Health and Wellness: Definition und Abgrenzung

Personal Health and Wellness umfasst konsumentenorientierte IoT-Systeme zur Erfassung von Aktivität, Schlaf, Puls, Training oder Wohlbefinden. Im Mittelpunkt steht die **Selbstbeobachtung** und Alltagsunterstützung durch eigenverantwortliche Gesundheitsförderung.

Gehört dazu

Fitness-Tracker und Smartwatches (z. B. Fitbit, Apple Watch), smarte Waagen, Schlafanalysegeräte, App-verbundene Sportgeräte und Stressmonitore für den Alltag.

Abgrenzung von Connected Healthcare

Der entscheidende Unterschied zu Kategorie 6 liegt in der medizinischen Einbindung. Bei Personal Health and Wellness gibt es **keine ärztliche Supervision**, keine klinische Diagnose und keine therapeutische Bindung. Das Gerät dient der persönlichen Motivation – nicht der medizinischen Behandlung.



Personal Health and Wellness: Daten, Ziele und Kommunikationsbedarf

Erhobene Daten

Schrittzahl, Herzfrequenz in Ruhe und unter Belastung, Schlafdauer und -qualität, Trainingsintensität, kalorischer Verbrauch, Stressindikatoren (z. B. Herzratenvariabilität) und zum Teil Hautleitfähigkeit.

Ziele der Auswertung

Verhaltensänderung durch Feedback, Steigerung des Gesundheitsbewusstseins, **Motivation zur regelmäßigen körperlichen Aktivität** und langfristige Prävention von Erkrankungen durch gesündere Lebensweise. Gamification.

Kommunikationsbedarf

Kleine, mobile und energiearme Geräte, die typischerweise über BLE mit dem Smartphone kommunizieren. Das Smartphone dient als Gateway in Cloud-Plattformen der Hersteller. Akkulaufzeit von Tagen bis Wochen ist marktentscheidend.

Connected Healthcare: Definition und Abgrenzung

Connected Healthcare umfasst im IoT-Kontext vernetzte medizinische Geräte und Systeme zur Beobachtung von Patientenzuständen außerhalb klassischer Klinikprozesse. Ein zentraler Teilbereich ist das **Remote Patient Monitoring (RPM)** – die Fernüberwachung medizinisch relevanter Vitalparameter.

Gehört dazu

Vernetzte Blutdruckmessgeräte, Blutzuckermessgeräte für Diabetespatienten, Herzmonitore, Implantate mit Datenübertragung und medizinisch zertifizierte Wearables unter ärztlicher Aufsicht.

Abgrenzung

Allgemeine Consumer-Fitness-Systeme ohne direkte medizinische Betreuung, klinische Diagnose oder Therapiebindung gehören in die Kategorie Personal Health and Wellness – nicht hierher. Der Unterschied liegt in der medizinischen Einbindung und Regulierung.



Spirometers



Blood Pressure



Blood Glucose



Weight Monitors

Retail und Inventory Management: Daten, Ziele und Kommunikationsbedarf

Erhobene Daten

Warenbestände (per RFID oder Kamera), Regalfüllstände, Temperaturen in Kühlmöbeln, Besucherorte und Verweilzeiten, Energieverbrauch der Filialinfrastruktur.

Ziele der Auswertung

Reduktion von Out-of-Stock-Situationen (leere Regale), weniger Verderb durch besseres Kühlkettenmanagement, effizientere Filialprozesse und datenbasierte Betriebssteuerung statt manueller Kontrolle.

Kommunikationsbedarf

Sensoren, Beacons und elektronische Regaletikett (Electronic Shelf Labels, ESL) sollen flexibel, kostengünstig und ohne feste Verkabelung betrieben werden. Typische Technologien: RFID, Bluetooth Low Energy (BLE), Wi-Fi und proprietäre Funklösungen für ESL.



Retail und Inventory Management: Rezipienten, Aktuatoren, Erfolg und Praxisbeispiel

Rezipienten der Daten

Filialleitung, Disposition und Einkauf, Technische Wartung, Marketing (Kundenverhalten) und die Lieferkette (automatische Nachbestellung).

Aktuatoren

Automatische Preisänderungen auf elektronischen Regaletiketten, Alarme bei Temperaturabweichungen, ausgelöste Nachbestellprozesse und Eingriffe in Kühlsysteme.

Erfolgskriterien

Bestände sind genauer sichtbar, Out-of-Stock-Quoten sinken, Verderb wird reduziert und operative Entscheidungen werden schneller und zuverlässiger getroffen.

Praxisbeispiel

VeloSiot dokumentiert reale Retail-IoT-Beispiele: velosiot.com – IoT in Retail



Öl und Gas: Definition und Abgrenzung

Die Öl-und-Gas-Industrie (englisch: **Oil & Gas**) gehört zur kritischen Energieinfrastruktur. IoT-Systeme in diesem Bereich umfassen die vernetzte Überwachung von Pipelines, Förderanlagen, Tanks und sicherheitskritischen Prozessen über zum Teil extreme Entfernungen und unter rauen Umgebungsbedingungen.

Typische Anwendungen

Pipeline-Monitoring auf tausenden Kilometern Länge, Leckerkennung, Fernüberwachung von Offshore-Plattformen, Tankniveauüberwachung, Korrosionsmonitoring und sicherheitskritische Überwachung von Druckbehältern.

Abgrenzung

Von allgemeiner Fabrikautomation unterscheidet diese Domäne der spezifische Infrastrukturtyp, die strenge Regulierung durch Behörden, das besonders hohe Sicherheitsniveau (Safety Integrity Level, SIL) und die extremen Umgebungsbedingungen.



<https://interaktiv.tagesspiegel.de/lab/globaler-pipeline-atlas-das-unsichtbare-netz-der-weltweiten-energieversorgungs/>

Öl und Gas: Daten, Ziele und Kommunikationsbedarf

Erhobene Daten

Druck, Durchfluss, Temperatur, Korrosionsgrad, Leckindikatoren (akustische Emissionen, Druckabfall), Standortdaten mobiler Einheiten, Gaskonzentrationen in der Umgebung und Umweltparameter.

Ziele der Auswertung

Gewährleistung von Sicherheit und Anlagenverfügbarkeit, Reduktion kostspieliger Vor-Ort-Inspektionen in abgelegenen Gebieten, frühzeitige Erkennung kritischer Zustände und Vermeidung von Umwelt-katastrophen durch Lecks oder unkontrollierte Freisetzung.

Kommunikationsbedarf

Besonders herausfordernd: Anlagen sind oft abgelegen (Arktis, Offshore, Wüste), weit verteilt und rauen Bedingungen ausgesetzt. Satellitenkommunikation, Mobilfunk in Kombination mit lokalen Mesh-Netzen und kabelgebundene Feldbusse sind im Einsatz.



Telekommunikation: Definition und Abgrenzung

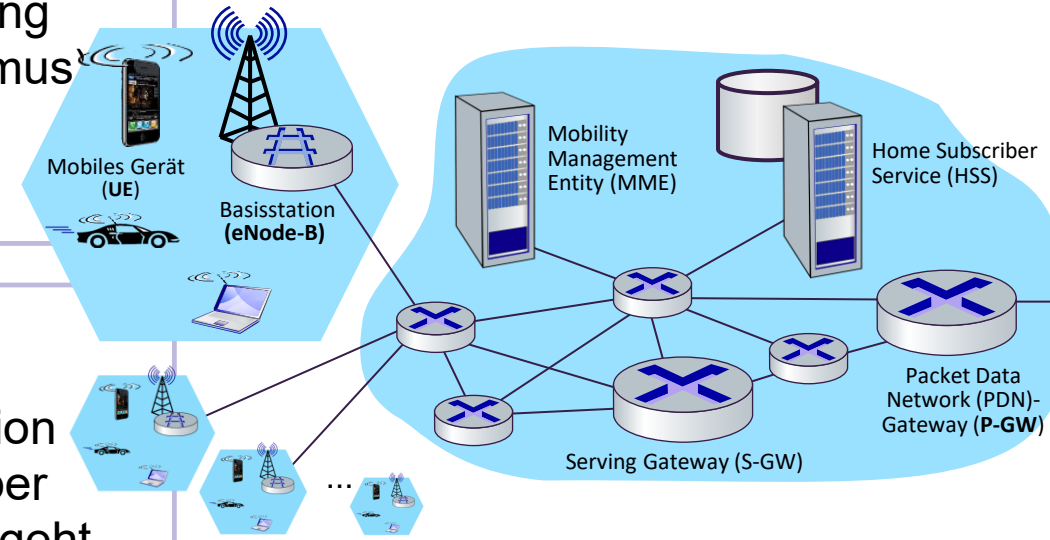
In dieser Kategorie wird IoT nicht nur *über* Telekommunikationsnetze realisiert, sondern **zur Überwachung und Optimierung der Netzinfrastruktur selbst** eingesetzt. Telekommunikationsunternehmen (Telcos) betreiben tausende geografisch verteilter Netzknoten, die selbst zu IoT-Endpunkten werden.

Gehört dazu

Monitoring von Funkmasten (Base Stations), Energieversorgung der Infrastruktur, Standortbedingungen (Temperatur, Vandalismus), Ausfallüberwachung, Quality-of-Service (QoS)-Daten und automatisches Network-Fault-Management.

Abgrenzung

Allgemeine Kundenanwendungen, bei denen Telekommunikation nur als Transportmedium dient – etwa ein Smart Meter, das über NB-IoT kommuniziert – gehören nicht in diese Kategorie. Hier geht es ausschließlich um die Infrastruktur des Netzbetreibers selbst.



Smart Grid / Energy Management: Definition und Abgrenzung

Ein **Smart Grid** (intelligentes Stromnetz) beschreibt die digitale Vernetzung von Energieerzeugung, Netzbetrieb, Speicherung und Verbrauch. Im Unterschied zum klassischen Stromnetz fließen im Smart Grid Informationen **bidirektional** – von der Erzeugung zum Verbrauch und zurück.

Wichtige Unterbereiche

Smart Metering (intelligente Zähler), Lastmanagement, automatische Fehlererkennung und Netzschnittung sowie die Integration dezentraler Einspeiser wie Photovoltaik oder Windkraft.

Ziel: ~240 Volt bei 50Hz Wechselspannung

Abgrenzung

Reine lokale Gebäudesteuerung – etwa ein smartes Thermostat ohne Anbindung an Netzbetreiber oder Messinfrastruktur – ist davon abzugrenzen. Entscheidend ist der Bezug zum Energienetz und zur Messinfrastruktur.

Smart Grid: Rezipienten, Aktuatoren, Erfolg und Praxisbeispiel

Rezipienten der Daten

Netzbetreiber (Distribution System Operators, DSO), Energieversorger, Messstellenbetreiber und zum Teil Endkunden über transparente Verbrauchsdarstellungen in Apps oder Portalen.

Aktuatoren

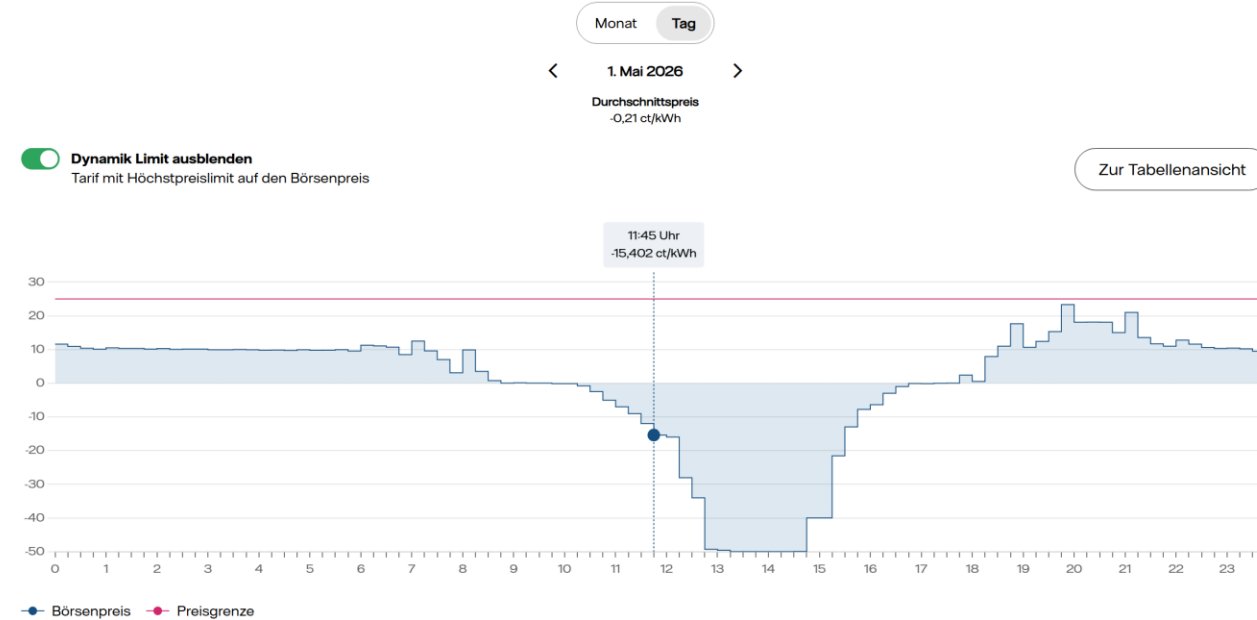
Laststeuerung (z. B. Abschalten steuerbarer Verbraucher), Tarifsignale, automatische Schalthandlungen im Netz und Netzsicherstellungen zur Fehlerbehebung.

Erfolgskriterien

Stabiler Netzbetrieb trotz mehr erneuerbaren Energien, Verbrauchstransparenz und effizientere Nutzung bestehender Netzinfrastruktur ohne Netzerweiterung.

Praxisbeispiel

Durch Photovoltaik und Windenergie hohe Spannungs-/Kapazitätswechsel.
Redispatch: Mittags teure Abgabe von Überschuss an Nachbarländer, abens teurerer Rückkauf. Doppelte Kosten.



Preise in ct/kWh: [EPEX-Spot DE](#) für reine Energie ohne Zuschläge und ohne Grünstromzertifikate. Für die angezeigten Werte übernimmt Vattenfall keine Gewähr.

<https://www.vattenfall.de/strom/tarife/oekostrom-dynamik-boersenpreise>

Vehicular Networks / V2X: Definition und Abgrenzung

Vehicle-to-Everything (V2X) bezeichnet die Kommunikation von Fahrzeugen mit ihrer Umgebung. Der Begriff fasst mehrere Kommunikationsrichtungen zusammen, die gemeinsam ein vernetztes Verkehrssystem ermöglichen.



Vehicle-to-Vehicle (V2V)

Fahrzeuge tauschen direkt Informationen aus, z. B. Bremswarnung oder Kollisionswarnung.



Vehicle-to-Infrastructure (V2I)

Fahrzeuge kommunizieren mit Ampeln, Schildern und Straßensendern.



Vehicle-to-Network (V2N)

Fahrzeuge sind mit Verkehrsmanagementzentralen und Serviceplattformen verbunden.

Reine Bordelektronik ohne externe Vernetzung – etwa das interne Steuergerät (Electronic Control Unit, ECU) eines Fahrzeugs – gehört **nicht** zu dieser Kategorie.

V2X: Daten, Ziele und Kommunikationsbedarf

Erhobene Daten

Position (GNSS-Koordinaten), Geschwindigkeit, Fahrtrichtung (Heading), Bremsstatus, Signalphasen von Ampeln, Straßenzustand und sicherheitskritische Warnmeldungen.



Ziele der Auswertung

Erhöhung der Verkehrssicherheit durch frühere Reaktion auf kritische Situationen, bessere Verkehrscoordination zur Reduktion von Stau und – langfristig – die Ermöglichung des automatisierten und autonomen Fahrens. Keine Zeit für Cloud-Intervention → Lokale Entscheidungen.

Kommunikationsbedarf

Besonders anspruchsvoll, da Mobilität, sehr geringe Latenz und hohe Zuverlässigkeit gleichzeitig gefordert sind. Relevante Technologien sind WLAN-basiertes Dedicated Short Range Communication (DSRC) sowie zelluläres V2X (C-V2X) auf Basis von 4G/5G.

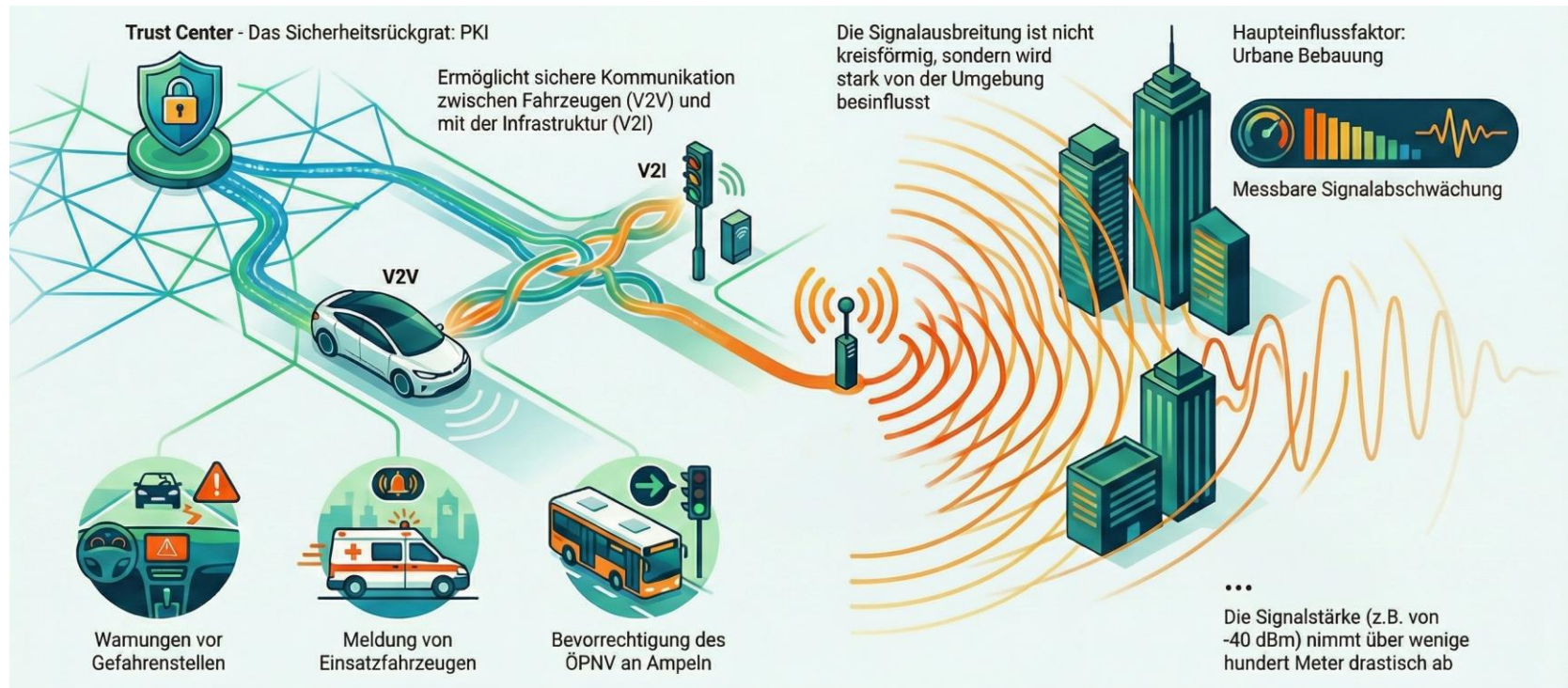
V2X: Rezipienten, Aktuatoren, Erfolg und Praxisbeispiel

Rezipienten der Daten

Fahrerassistenzsysteme (Advanced Driver Assistance Systems, ADAS), Verkehrsmanagementzentralen, Flottenbetreiber und perspektivisch vollautonome Fahrsysteme ohne menschlichen Fahrer.

Aktuatoren

Akustische und visuelle Fahrerwarnungen, Eingriffe in Fahrerassistenzfunktionen (z. B. automatisches Bremsen), adaptive Signalbeeinflussung bei Ampeln und Streckenfreigaben.



Smart Home: Definition und Abgrenzung

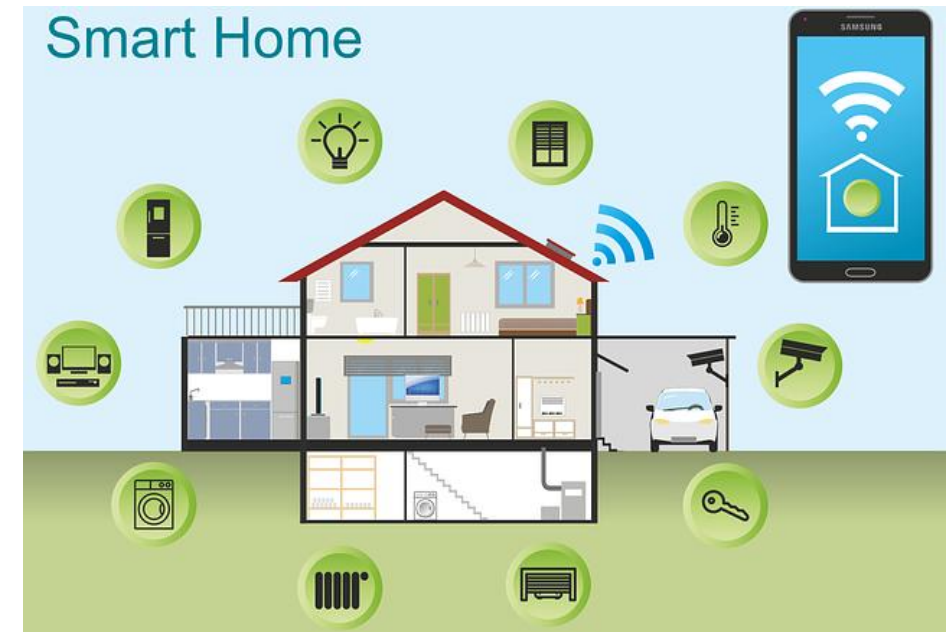
Smart Home bezeichnet vernetzte Systeme in Wohnumgebungen, die Funktionen wie Beleuchtung, Heizung, Sicherheit, Haushaltsgeräte oder Mediensteuerung automatisieren und fernsteuerbar machen. Der Betrieb erfolgt sowohl lokal – etwa durch Zeitpläne oder Bewegungssensoren – als auch über cloudgestützte Dienste per App oder Sprachassistent.

Gehört dazu

Smarte Thermostate, Beleuchtung, Türschlösser, Sicherheitskameras, Steckdosen, Haushaltsgeräte und Sprachassistenten in privaten Wohnräumen.

Gehört nicht dazu

Sobald aus der Wohnung ein gewerbliches oder institutionelles Gebäude wird, fällt die Verwaltung in den Bereich Facility Management. Die Grenze liegt am Nutzungstyp und an der organisatorischen Verantwortung.



Temperatur, Luftfeuchtigkeit,
Präsenz (Bewegung),
Tür- und Fensterstatus,
Energieverbrauch pro Gerät,
Gerätezustände, Lichtstärke
und Kameradaten.

Komforterhöhung durch Automatisierung, Energieeinsparung, Sicherheit der Wohnung, Fernzugriff und Benachrichtigungen bei Ereignissen.

Drahtlose Kommunikation ist besonders wichtig, da Bestandswohnungen nur begrenzt nachträglich verkabelt werden können. Viele Sensoren arbeiten batteriebetrieben und müssen energiearm kommunizieren. Typische Standards sind Wi-Fi, Zigbee und Bluetooth Low Energy (BLE).



Büro- und Facility Management: Definition und Abgrenzung

Facility Management im IoT-Kontext umfasst vernetzte Systeme zum Betrieb und zur Optimierung größerer Gebäude und Liegenschaften wie Bürogebäude, Einkaufszentren, Universitäten oder öffentliche Einrichtungen.

Typische Bereiche

Raumbelegungsmessung, Energieverbrauchsüberwachung, Klimatisierung und Heizungssteuerung, Zugangskontrolle, Sicherheitssysteme und die technische Wartungsplanung von Gebäudeanlagen (z. B. Aufzüge, Lüftungsanlagen).

Abgrenzung vom Smart Home

Facility Management unterscheidet sich vom Smart Home durch den **Maßstab** (viele Räume, Stockwerke, Gebäude), die **organisatorische Verantwortung** (professioneller Betrieb) und den **Gebäudetyp** (gewerblich, institutionell, öffentlich).



Smart Home: Rezipienten, Aktuatoren, Erfolg und Praxisbeispiel

Rezipienten der Daten

Primär die Bewohnerinnen und Bewohner über Apps und Dashboards. Teilweise auch Plattformbetreiber (für aggregierte Nutzungsanalysen) und Energiedienstleister (Lastmanagement).
(Fernauslesbare Heizkostenverteiler ab 2027 Pflicht)

Aktuatoren

Thermostate, Lichtschalter, Rollläden, Türschlösser, schaltbare Steckdosen, Alarmanlagen und smarte Haushaltsgeräte.

Erfolgskriterien

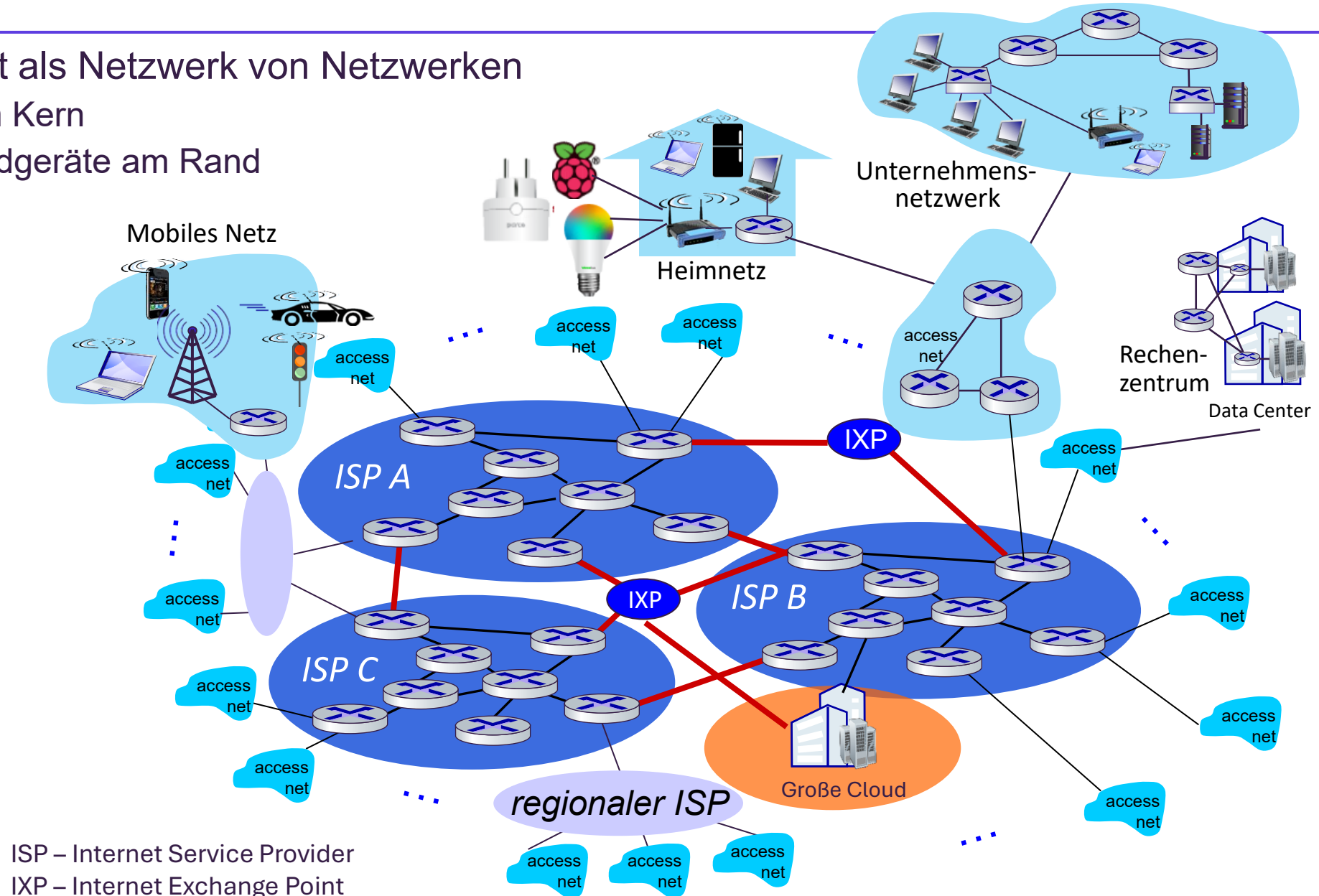
Stabile Funktion im Alltag, messbarer Nutzen (z. B. Energieeinsparung), geringe Bedienlast und hohe Ausfallsicherheit.



Internetstruktur: Ein Netzwerk von Netzwerken

Das Internet als Netzwerk von Netzwerken

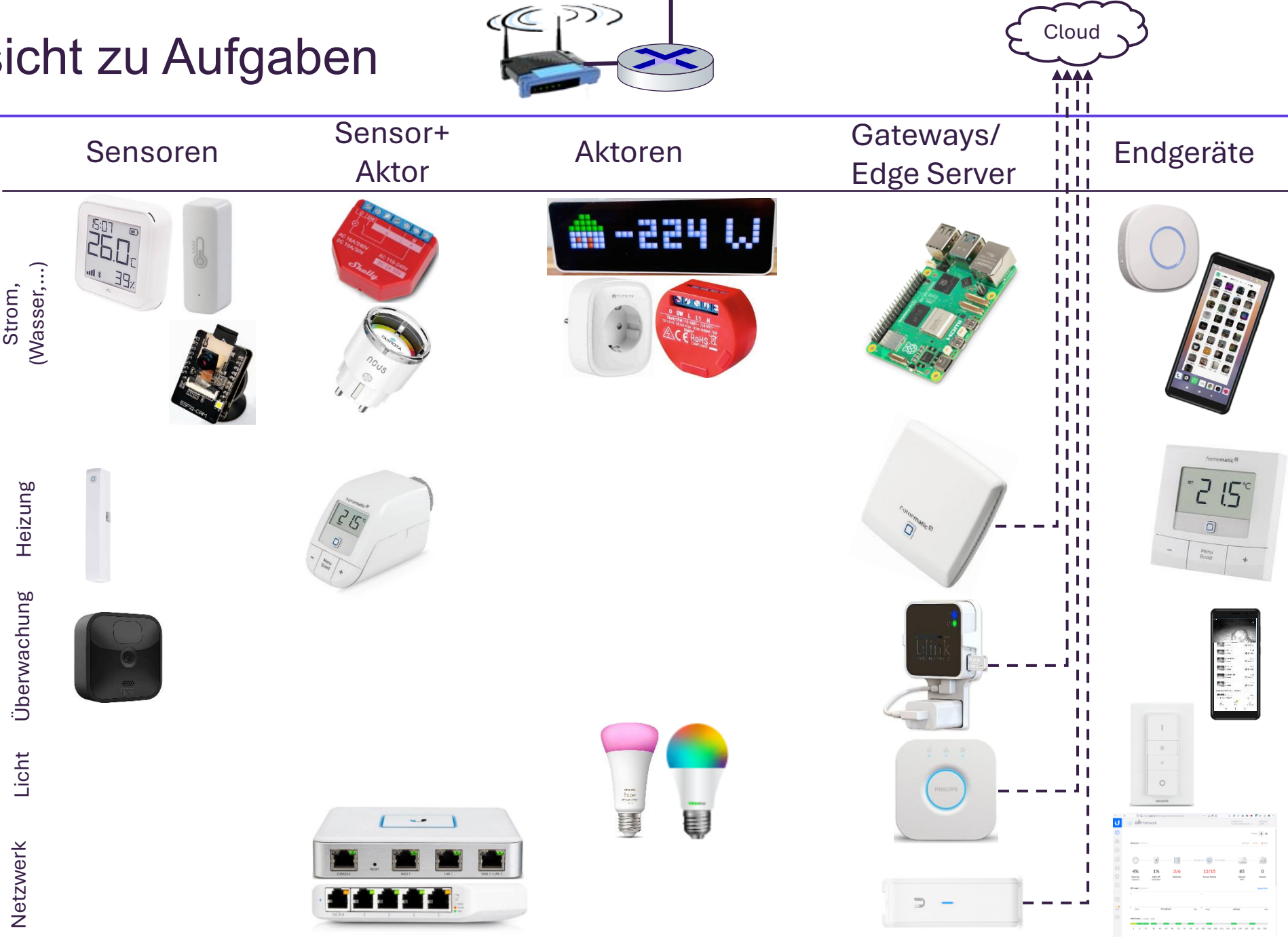
- Router im Kern
- Edge: Endgeräte am Rand



Smart Home Beispiel



Übersicht zu Aufgaben



Komponenten einer Edge – IoT – Infrastruktur

Sensoren

- Geräte, die Informationen erfassen und versenden
- Liefern Datenbasis für alle weiteren Analysen und Automatisierungen
- Spezialfall Smart Sensor: Vorverarbeitung der Daten, z.B. mittels Datenauswertung im Gerät

Aktoren

- Reagieren auf Befehle und bewirken direkt Veränderungen (z.B. durch Schalten, Steuern oder Regeln)
- Oft kombiniert mit Sensorfunktionalität (z.B. schaltbare Steckdosen mit Verbrauchsmessung)

Gateways / Edge Server:

- Lokales Gerät zur Datenverarbeitung und -weiterleitung sowie der Steuerung der Aktuatoren
- Bindeglied zwischen den Sensoren/Aktuatoren und einer eventuell übergeordneten Cloud
- Spezialfall – Hubs: Brücken zwischen unterschiedlichen Kommunikationsprotokollen (z.B. ZigBee- zu Wi-Fi)

Endgeräte

- Zur Interaktion des Benutzers mit dem IoT-System, z.B. zur Überwachung und Steuerung

Clouddienste

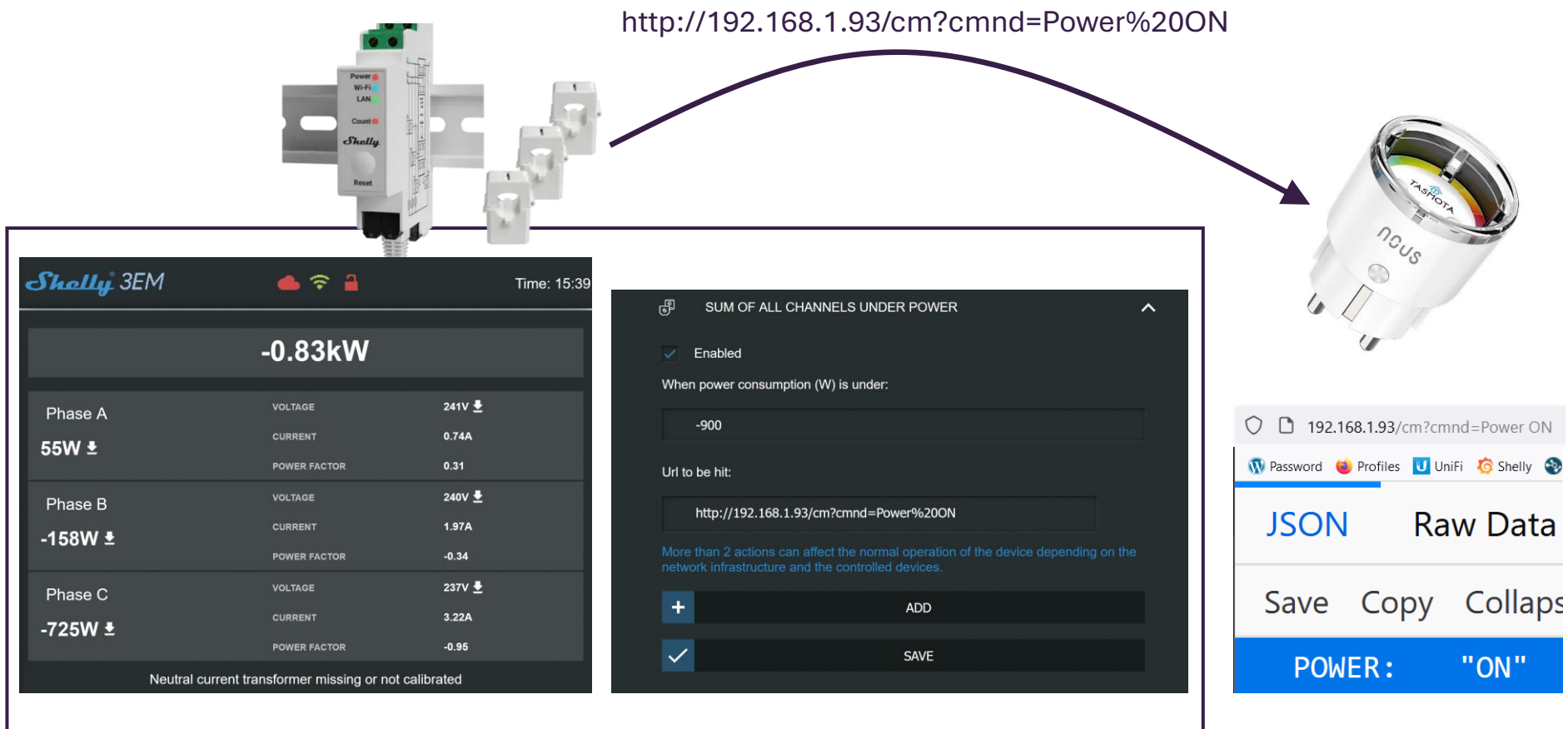
- Oft Element einer IoT Infrastruktur – „globaler Edge Server“
- Speichert Daten langfristig, aus vielen Quellen
- Führt komplexere Analysen durch und steuert Geräte von überall

Kommunikationsmuster – Direkt – Device zu Device

Direkte Kommunikation: Sensor sendet Aktuator direkt eine Nachricht

- Beispiel: Stromzähler misst negativen Verbrauch (z.B. → Photovoltaik), ruft dann konfigurierte URL auf (smarte Steckdose mit Warmwasser-Heizung)

Datenauswertung im Device



Kommunikationsmuster – Direkt – Device zu Gateway

Gateways / Edge Server: Verwalten Informationen von mehreren Geräten

- Aufgabe: Auswertung, (Analyse), Bereitstellung für Endgeräte
- Beispiel:
 - Blink Camera (Device) wird durch Bewegungsmelder ausgelöst, filmt Bewegung, zeichnet Abschnitt auf → Sendet Video-Snippet an Blink Sync Module (Gateway)
 - Blink Sync Module (mit USB Stick) speichert Video. Zugreifbar über App.

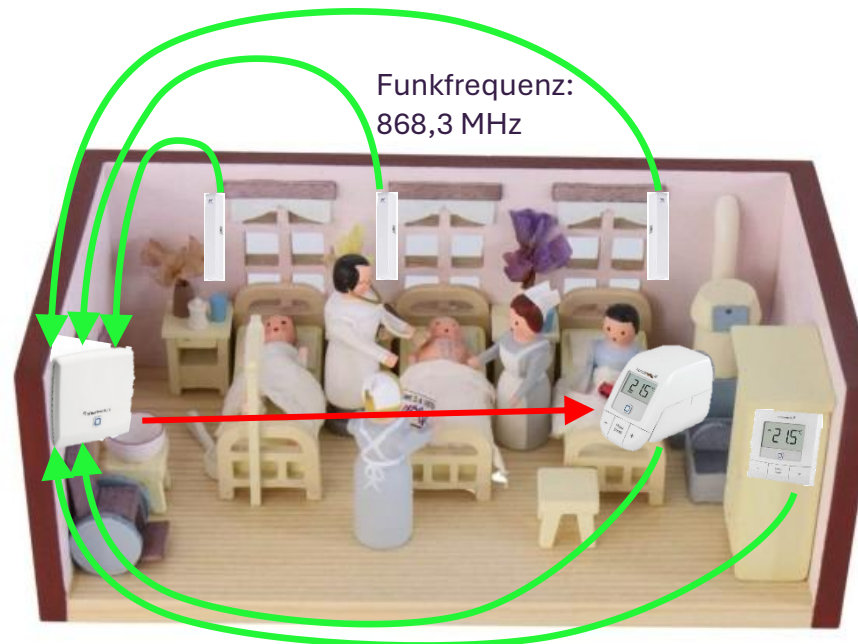
Video per 2,4 GHz WLAN
Befehle per 900-MHz Blink-Protokoll



Kommunikationsmuster – Direkt – Devices zu Gateway

Gateways / Hub: Kommuniziert mit mehreren Geräten in versch. Netzen

- Aufgabe: Sammelt und analysiert Daten, überträgt Nachrichten von einem Netz (ZigBee, Wifi, ...) in andere. Kontrolliert die Aktuatoren.
- Beispiel:
 - Fenstersensoren, Thermostat (Temperatur) und Regler (Sollvorgabe) liefern Daten
 - Homematic IP Thermostat wird von Homematic Access Point aktiviert, wenn:
 - Temperatur im Raum niedriger als Solltemperatur am Regler UND Fenster sind geschlossen

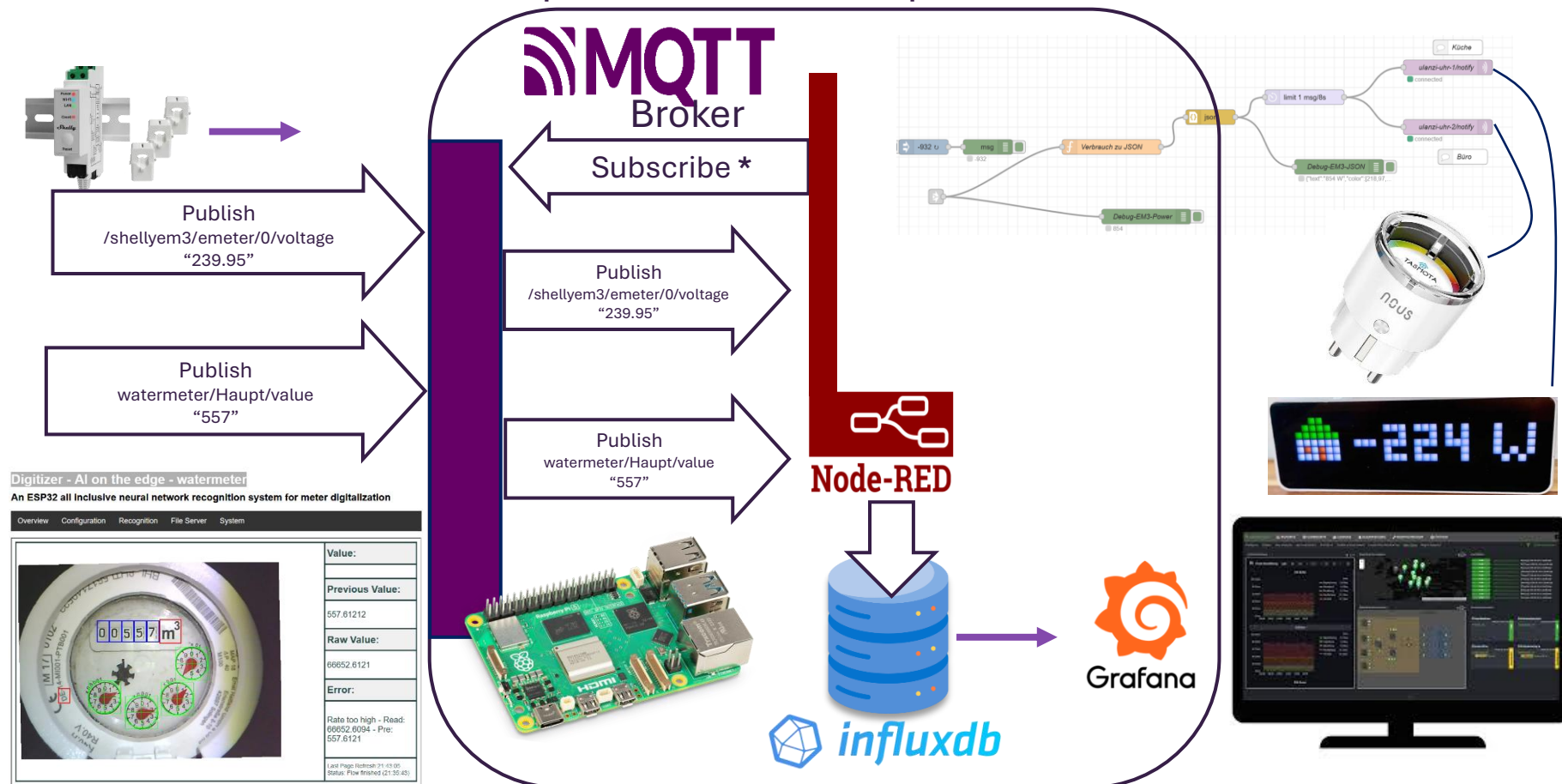


Kommunikationsmuster – Publish / Subscribe, Message Queue

Message Queuing Telemetry Transport – MQTT

Pub / Sub: Verteilt Msgs vom Topic X an alle Registrierten für Topic X

- Devices “publishen” ihre Daten an den Edge Server unter “Topics”
- Interessierte “subscriben” sich auf Topics, werden über Updates informiert

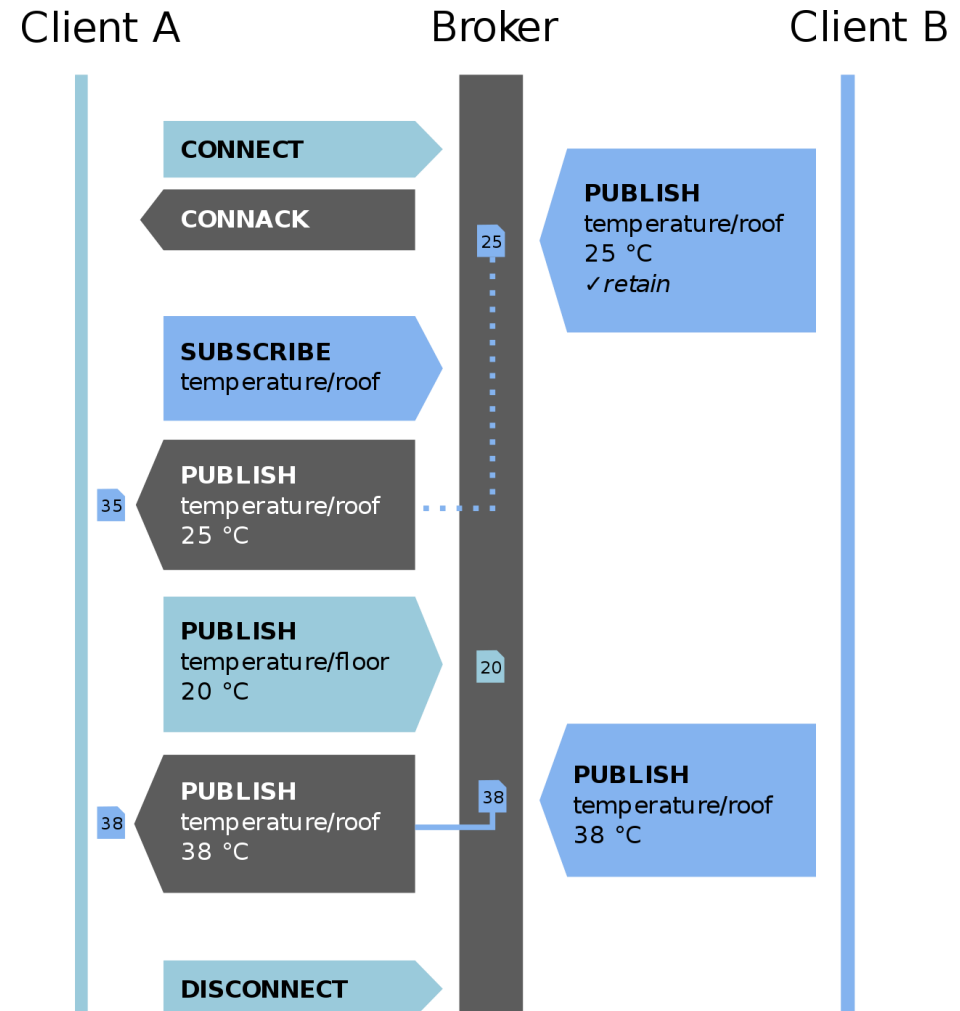


Message Queuing Telemetry Transport – MQTT

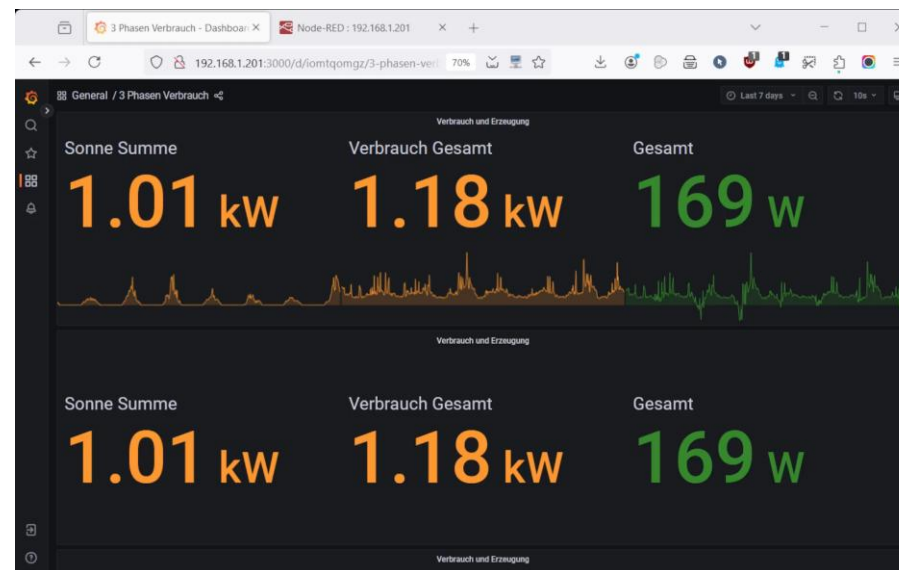
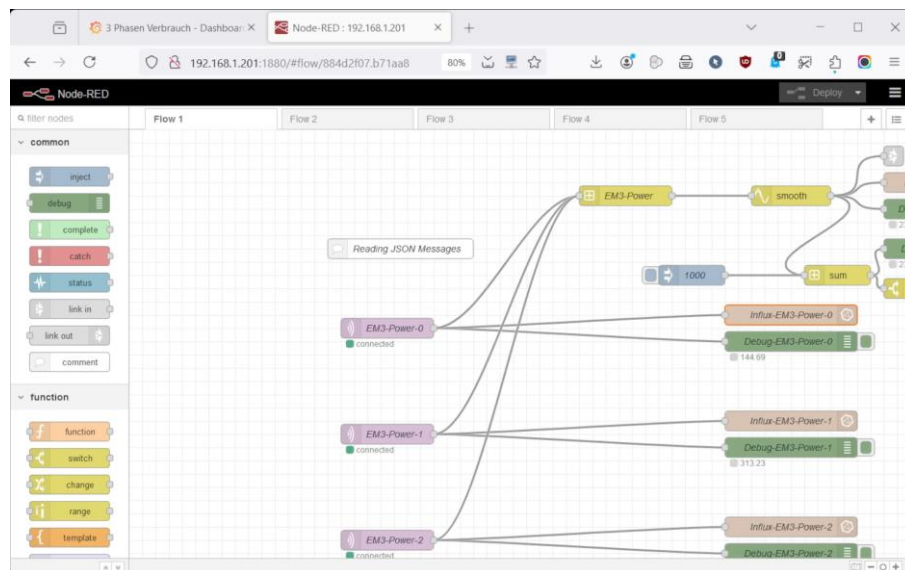
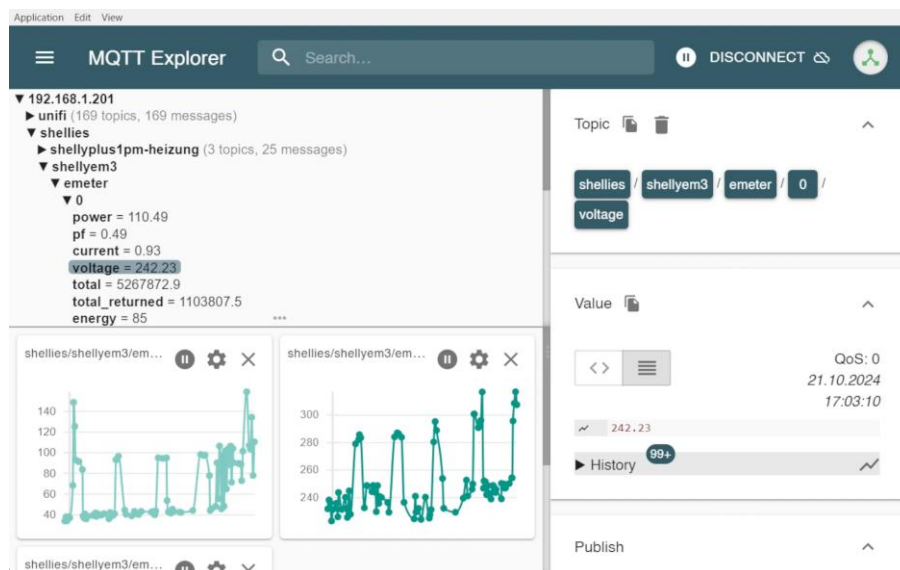
- Ist ein Publish / Subscribe Dienst
 - Verteilt Nachrichten vom Typ X an alle Registrierten für Typ X
- Erfordert einen Broker (Vermittler)
 - Für die Verwaltung der Subscriptions
 - Für die Weiterleitung der Nachrichten
- Message Typen
 - Connect / Disconnect / Publish
- Dienstgüte
 - Höchstens einmal
 - Mindestens einmal
 - Genau einmal

Andere Anwendungen

- XMPP – Extensible Messaging and Presence Protocol
- ...



Beispielfunktionalität eines Edge – Servers



Internet der Dinge – Internet of Things: IoT

Wir wollen unsere Geräte (Sensor, Mikrocontroller, PCs...) miteinander verbinden.

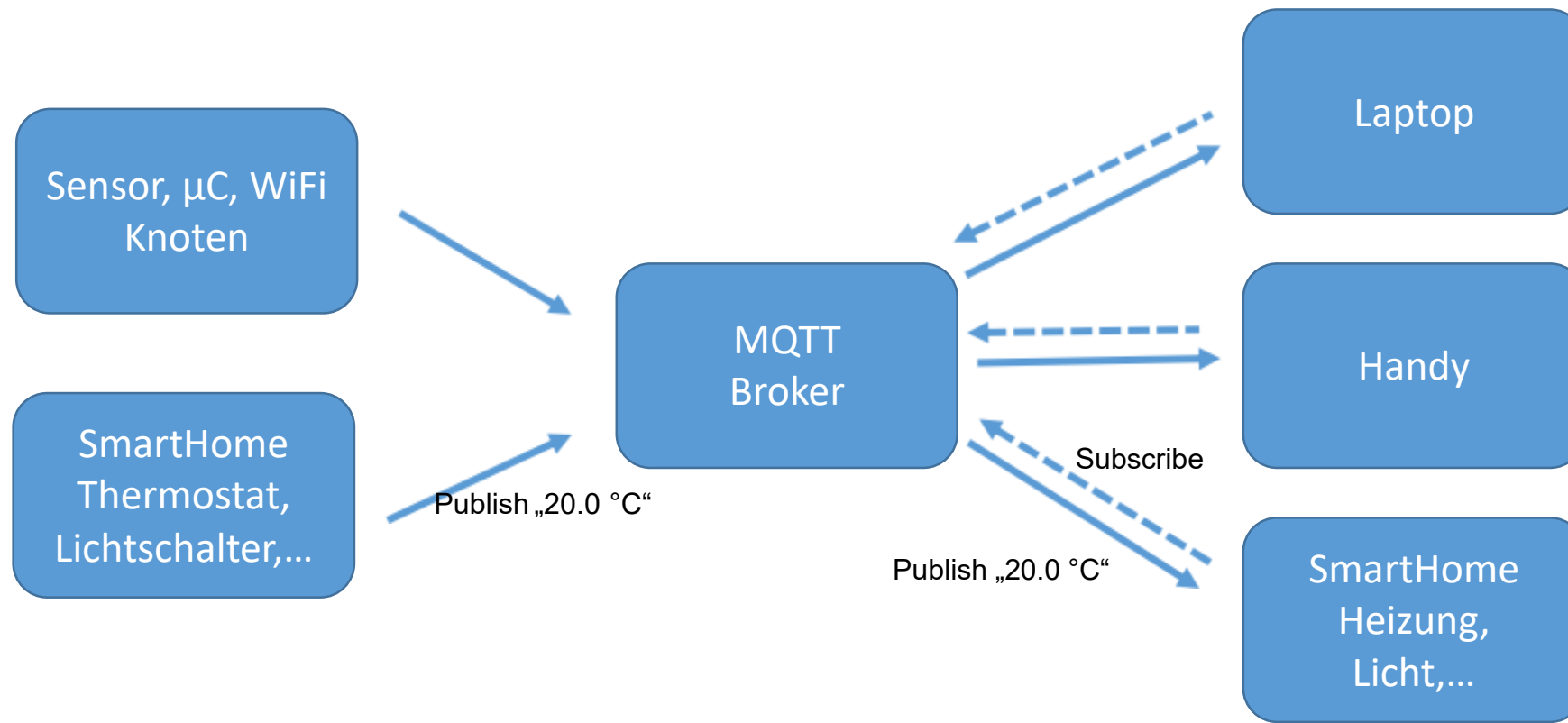
Wie machen wir das?

Bausteine für das IoT:

- MQTT –Message Queue TelemetryTransport
- Node-Red–<https://nodered.org/about/>
- JSON –JavaScript Object Notation
- openHAB–open Home Automation Bus
- u.v.m...

MQTT – Message Queue Telemetry Transport

MQTT: Publish/Subscribe



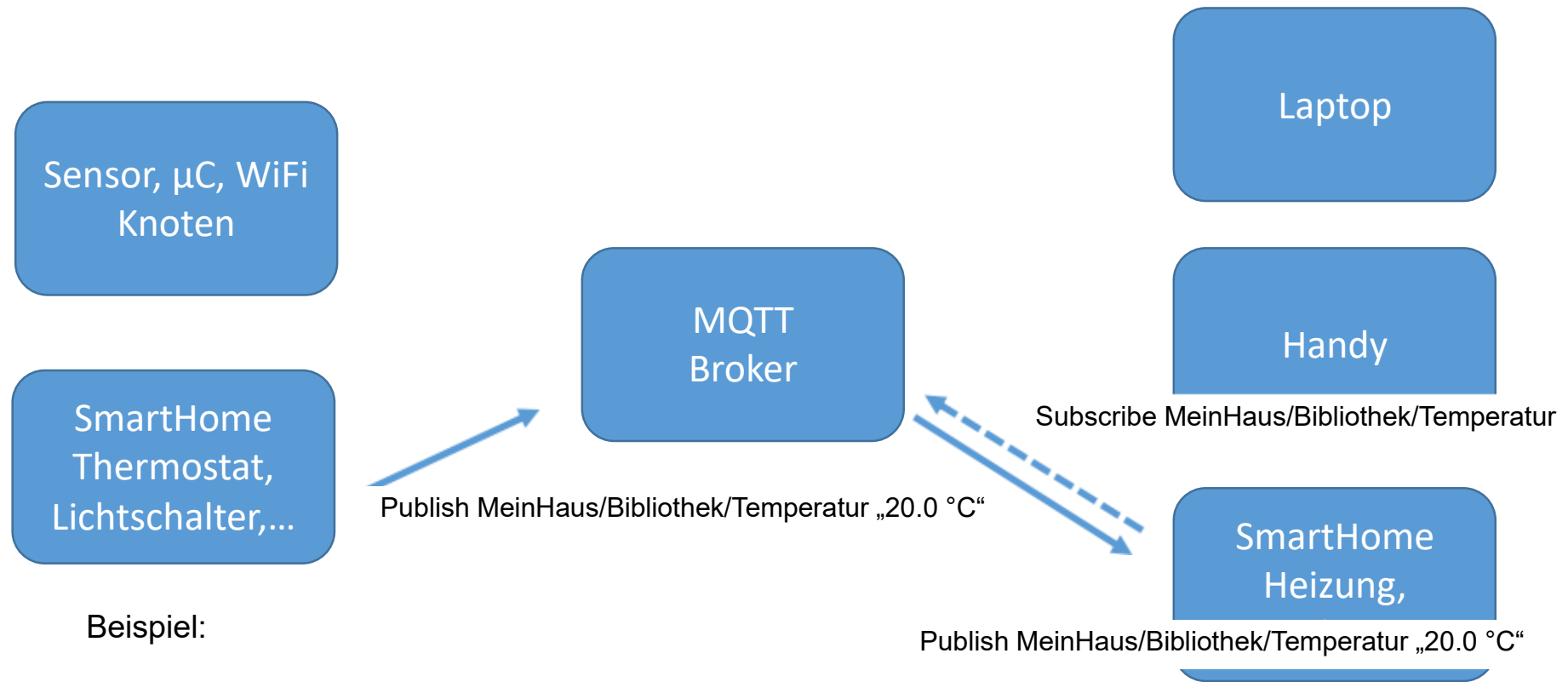
Im Gegensatz dazu HTTP: Request/Response

Bild nach: Walter Trokan, Das MQTT-Praxisbuch

MQTT – Message Queue Telemetry Transport

Adressierung über Topic („Betreff“)

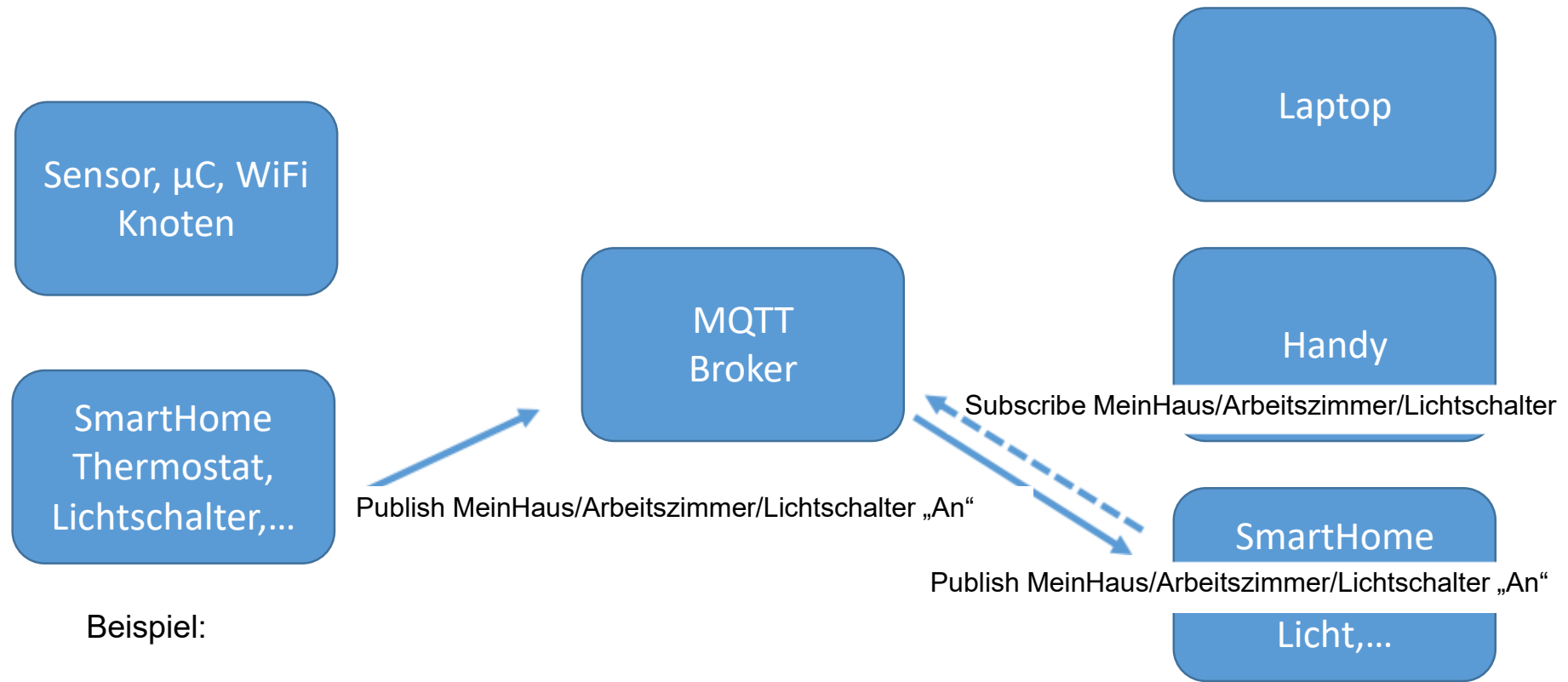
Nutzdaten als „Payload“



MQTT – Message Queue Telemetry Transport

Adressierung über Topic („Betreff“)

Nutzdaten als „Payload“

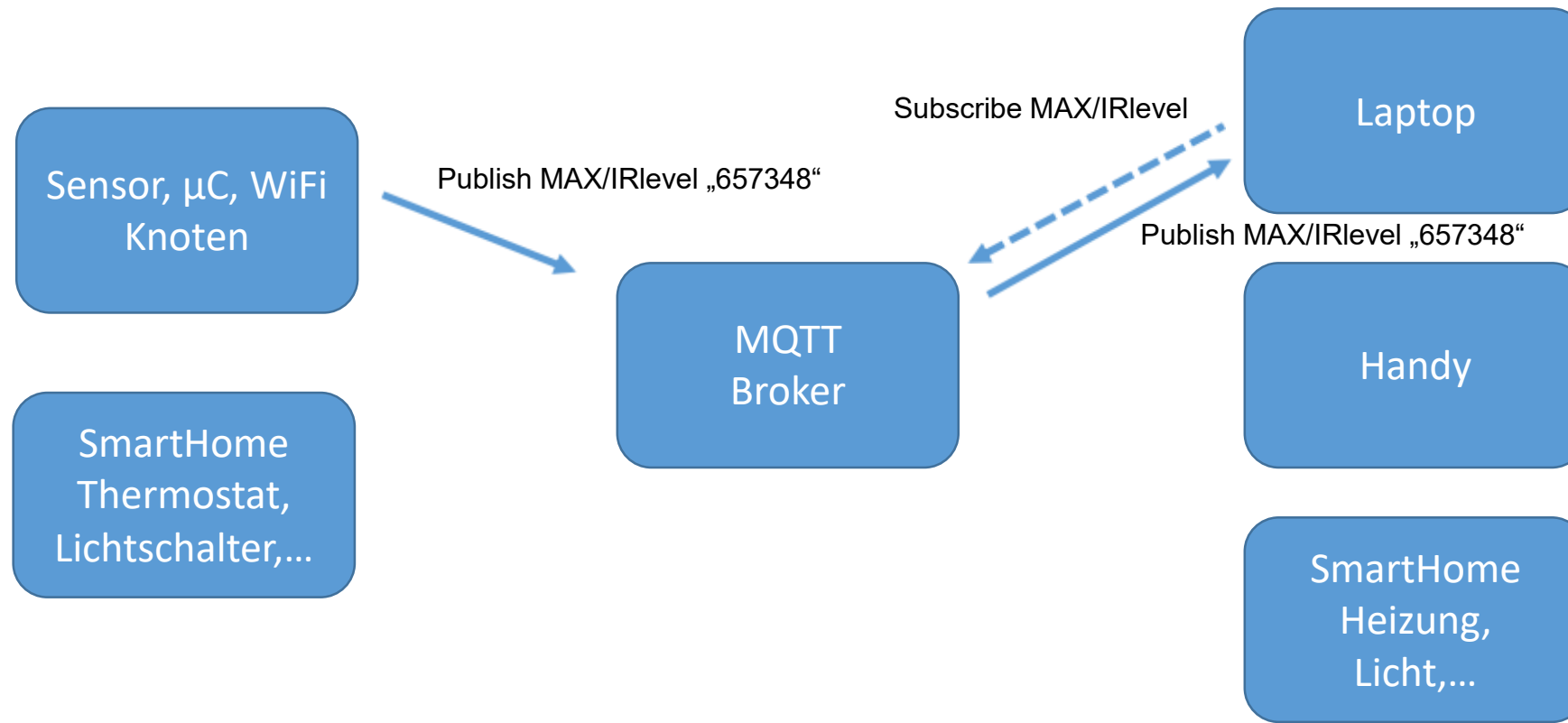


MQTT – Message Queue Telemetry Transport

„Wildcards“:

Single Level: MeinHaus/+/Temperatur

Multi Level: MeinHaus/#



MQTT – Qualitätssicherung

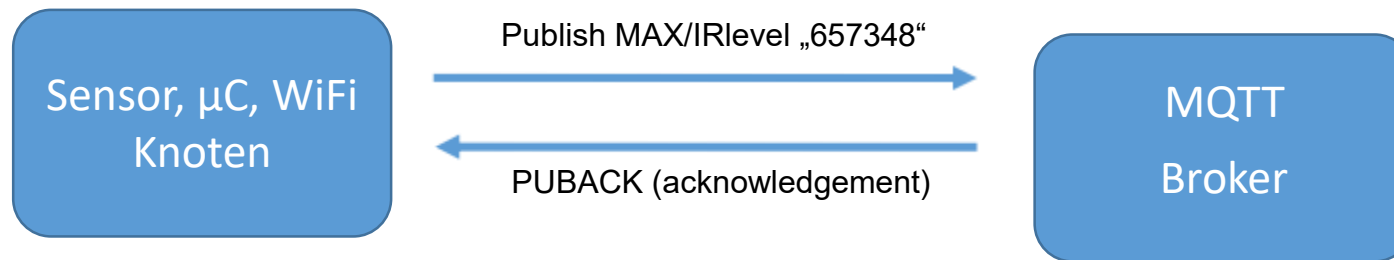
QoS0: Maximal einmal



Geringste Netzbelastung
Weder Empfangsbestätigung
noch Zwischenspeicherung beim Sender

MQTT – Qualitätssicherung

QoS1: Mindestens einmal

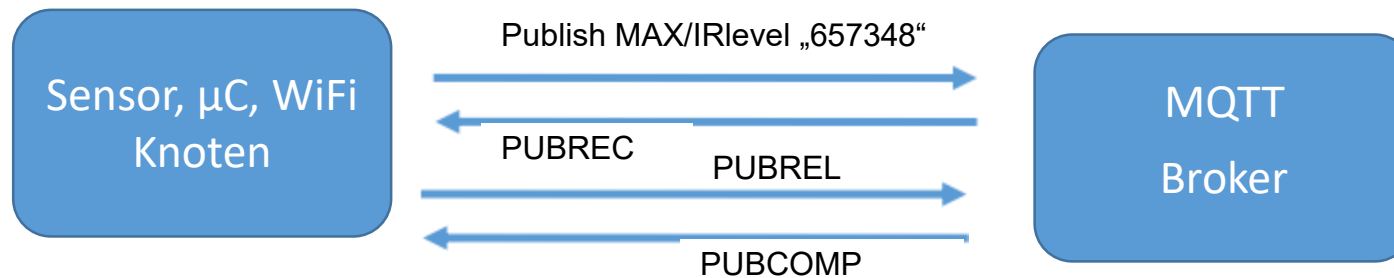


Moderate Netzbelastung
Empfangsbestätigung
Bis dahin Zwischenspeicherung beim Sender

Sender sendet bei ausbleibender Empfangsbestätigung mehrmals
Fehlermöglichkeit: z.B. Lichtschalter wird mehrmals betätigt

MQTT – Qualitätssicherung

QoS2: Garantiert nur einmal



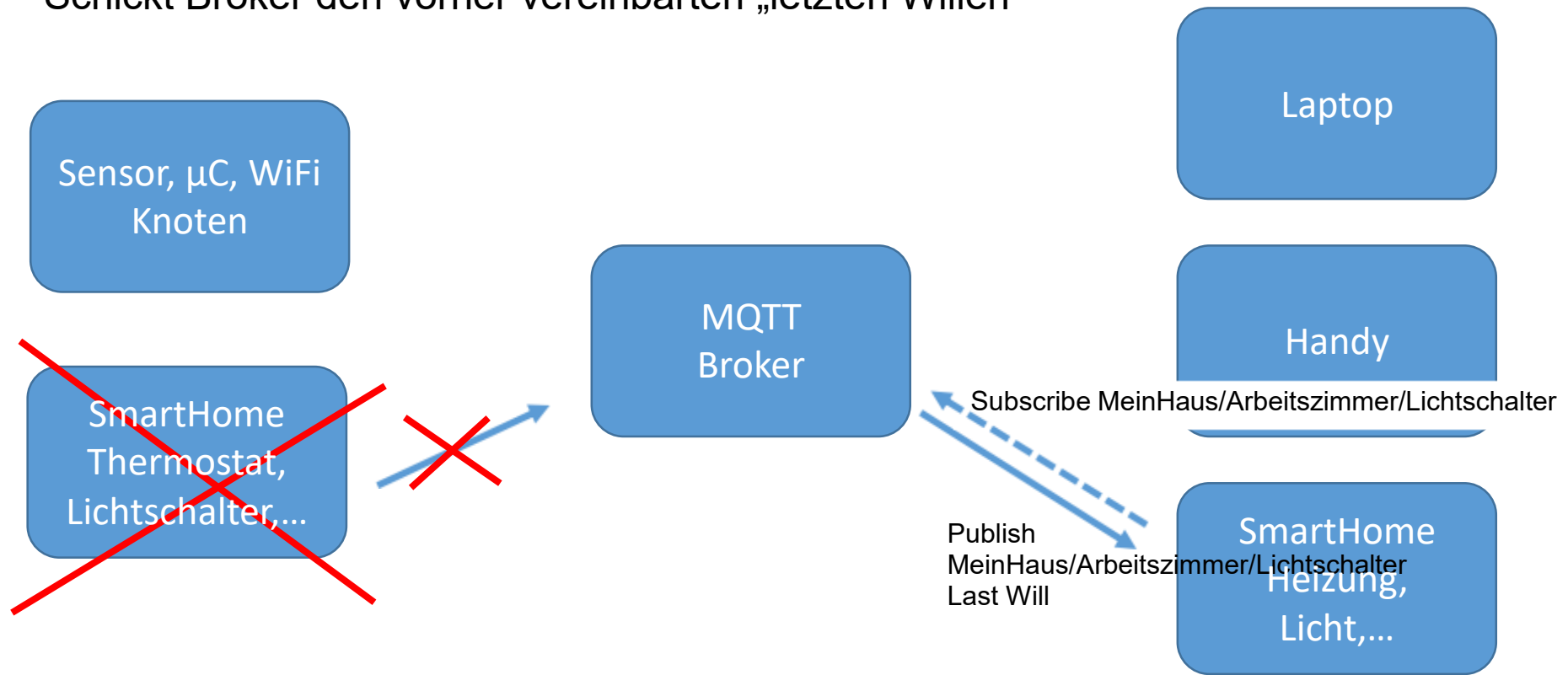
Moderate Netzbelastung
Empfangsbestätigung
Bis dahin Zwischenspeicherung beim Sender

Sender sendet bei ausbleibender Empfangsbestätigung mehrmals
Fehlermöglichkeit: z.B. Lichtschalter wird mehrmals betätigt

MQTT – LetzterWille/ Testament

Wenn Sensor „abnibbelt“

Schickt Broker den vorher vereinbarten „letzten Willen“



MQTT

Ausprobieren! Auf wokwi.com/esp32

Online ESP32 Simulator – MQTT Weather Logger

<http://www.hivemq.com/demos/websocket-client/>

Client Tools: MQTT-Explorer, MQTTX

Broker URL: broker.mqttdashboard.com

Port: 1883 (Ist der Default Port)

Subscription: wokwi-weather

➔ Save

The screenshot shows the Wokwi website, an online ESP32 simulator. The header includes the Wokwi logo and the tagline "World's most advanced ESP32 simulator". Navigation links for "Discord Community", "LinkedIn Group", and "Pricing" are present. The main heading is "Online ESP32 Simulator" with the subtext "A faster way to prototype IoT projects with the ESP32 microcontrollers". Below this, a "Featured projects" section displays six project thumbnails: "WiFi Scanning", "NTP Clock", "ESP32 HTTP Server", "Joke Machine", "MicroPython OLED Display", and "MicroPython MQTT Weather Logger". Each thumbnail shows a code snippet and a visual representation of the project setup.

MQTT

Ausprobieren!:

Subscribe: wokwi-weather
Publish: wokwi-weather

„Hi, hier bin ich...“



Auf dem Dashboard nachsehen was angekommen ist

ESP32: WiFi Zugang

```
#include <WiFi.h>
```

```
const char* ssid = "Wokwi-GUEST";    // Dummy WLAN für IoT  
const char* password = " ... ";
```

```
WiFiClient espClient;
```

```
void setup() {  
    WiFi.begin(ssid, password);  
  
    while(WiFi.status() != WL_CONNECTED) {  
        delay(500);  
        Serial.print(".");  
    }  
}
```

```
Serial.println("WiFi connected");  
Serial.println("IP address: ");  
Serial.println(WiFi.localIP());
```

ESP32 : Druck-Sensor BMP085 auslesen

```
#include <Adafruit_BMP085.h>

Adafruit_BMP085 bmp;

WiFiClient espClient;
float pressure;

void setup() {

    bmp.begin();
}

void loop() {

    pressure = bmp.readPressure();

}
```

ESP32 : MQTT initialisieren

```
#include <PubSubClient.h>

const char* mqtt_server = "broker.mqttdashboard.com";

PubSubClient client(espClient);
char msg[75];

void callback(char* topic, byte* payload, unsigned int length) { ... }

void setup() {
    client.setServer(mqtt_server, 1883); // Serveradresse, Port
    client.setCallback(callback);
    client.connect("ESP32", "myuser", "mypass")
                // Nutzernamen, Account-Name, Passwort
    client.subscribe("wokwi-weather"); // Abonniertes Topic
}

void loop() {
    client.loop();

    if (!client.connected()) { reconnect(); }
```


ESP32 : MQTT initialisieren

Diese Funktion wird ausgeführt, wenn eine Message zu einem subskribierten Topic ankommt:

```
void callback(char* topic, byte* payload, unsigned int length) { ... }
```

```
void loop() {
```

```
  client.loop();
```

```
  if (!client.connected()) { reconnect(); }
```

```
}
```

Diese MQTT Funktion prüft, ob eine Message angekommen ist und ruft gegebenenfalls die als callback() eingetragene Funktion auf.



Regelmäßig (z.B. alle 10 s oder 1 min) sollte die Verbindung zum Broker überprüft werden. reconnect() ist eine selbst zu definierende Funktion, die die Verbindung (wie in setup() {...}) wieder aufbaut.



ESP32 : MQTT publish

```
#include <stdio.h>
```

Schreibt formatiert die integer Zahl aus der Variablen pressure in den String msg (wie printf(„%d“,pressure), hier Begrenzung auf n = 75 Bytes).

```
snprintf(msg, 75, \"%d\", (int)(pressure);
```

```
client.publish(\" Sensors /Pressure\", msg);
```

Publish den String aus dem Char Array msg unter dem Topic „UliFeder/Pressure“.

Für Fließkommazahlen muss man auf dem ESP32 (oder Arduino) tricksen, weil die abgespeckten Versionen von sprintf(...) etc. keine Fließkommaformate unterstützen:

```
snprintf(msg, 75, \"%d.%02d\", (int)(pressure/100), (int)pressure%100);
```

```
client.publish(\"Sensors/Pressure\", msg);
```

ESP32 : MQTT subscribe

Einkommende Messages für ein subskribiertes Topic werden an die callback(...) Funktion weitergereicht und dort verarbeitet:

```
void callback(char* topic, byte* payload, unsigned int length) {  
    if ((char)payload[0] == 't') {  
        digitalWrite(BUILTIN_LED, LOW);    // Turn the LED on  
    } else {  
        digitalWrite(BUILTIN_LED, HIGH);   // Turn the LED off  
    }  
}
```

Hier wird (einfach) nur der erste Character msg[0] im msg Array mit „t“ verglichen. Ein „t“, „true“, „tobeornottobe“ schaltete die LED ein, ein „f“, „false“ aber auch ein „xbeliebig“ die LED aus.

MQTT – Wer holt die Daten ab?

Bis hier hin:

Der Sensor hat die Messdaten aufgenommen,
sich mit dem WLAN verbunden,
sich mit dem MQTT-Broker verbunden und
die Daten an Topics geschickt. (Können wir auf unserer App sehen)

Was kann man nun mit den Daten auf dem MQTT-Broker tun?
→ MQTT kann in eigene Programmen eingebunden werden,
um mit deren Hilfe die Messdaten darzustellen.

EIN Beispiel:

→ NodeRed

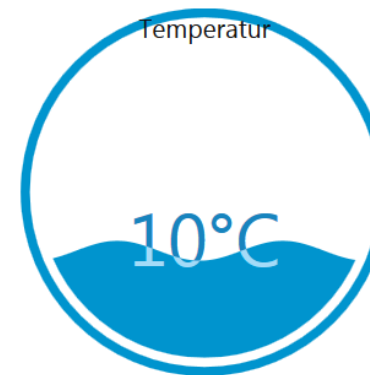
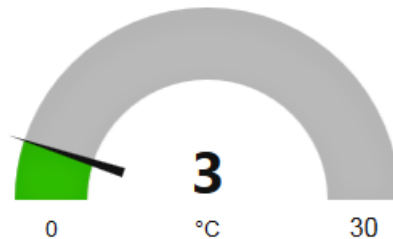
Node-Red

Flussdiagramm-basiertes Programmierwerkzeug (flow-based programming tool)

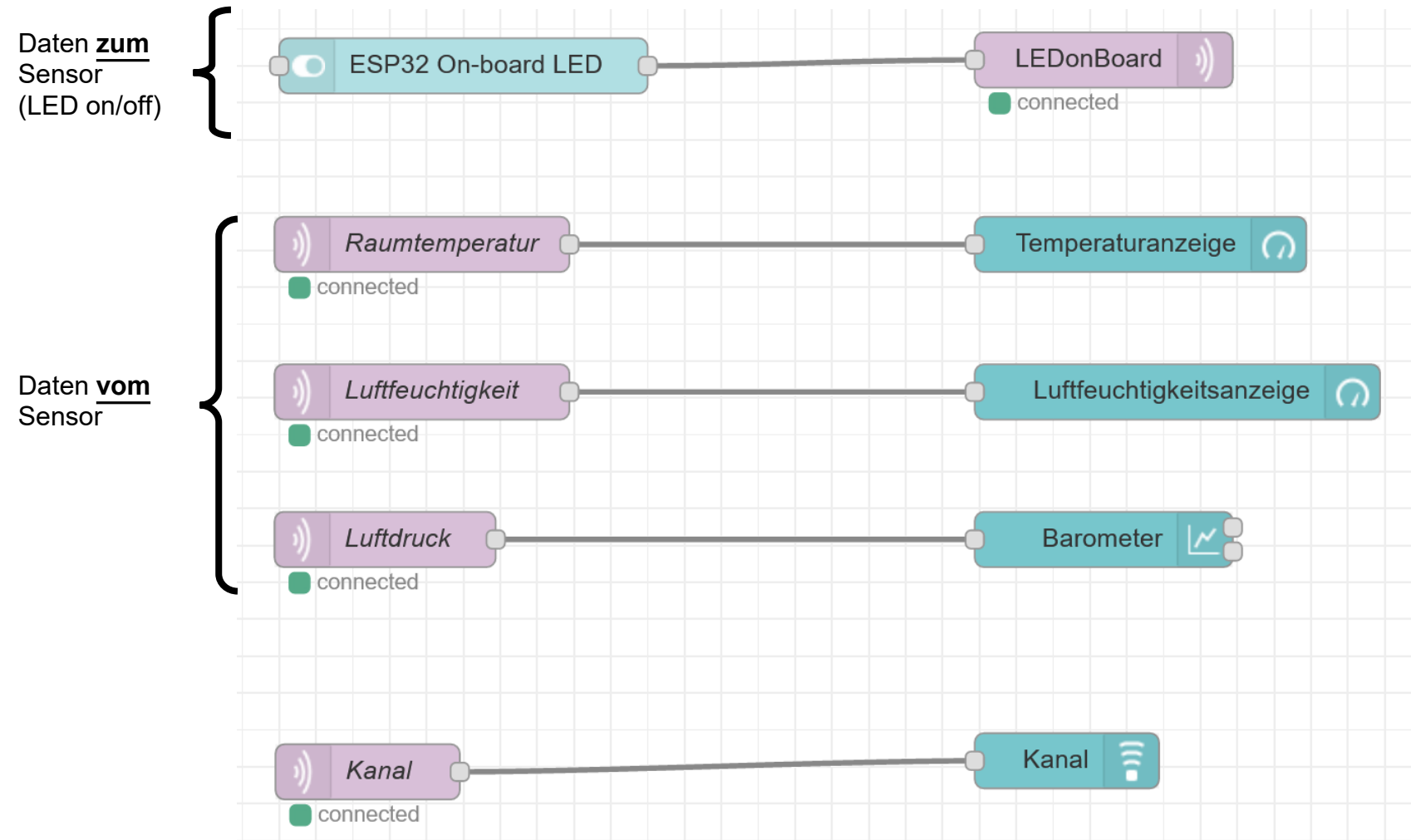


Es wird von einer Laufzeitumgebung (Programm Node-RED) ein „Dashboard“ erzeugt, das über einen Browser angezeigt werden kann:

aktuelle Temperatur in Chemnitz

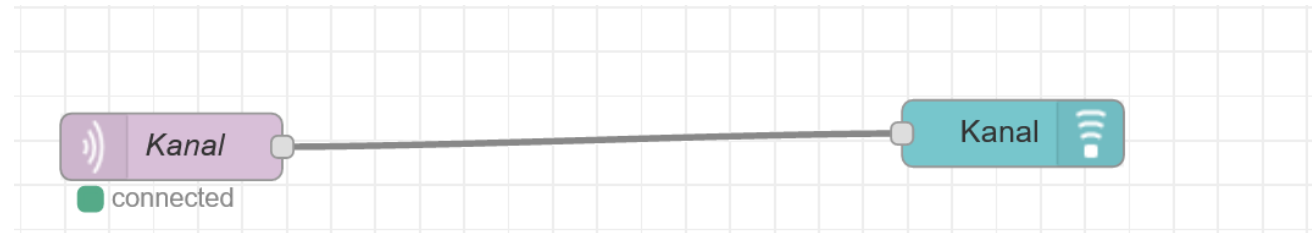


Node-Red



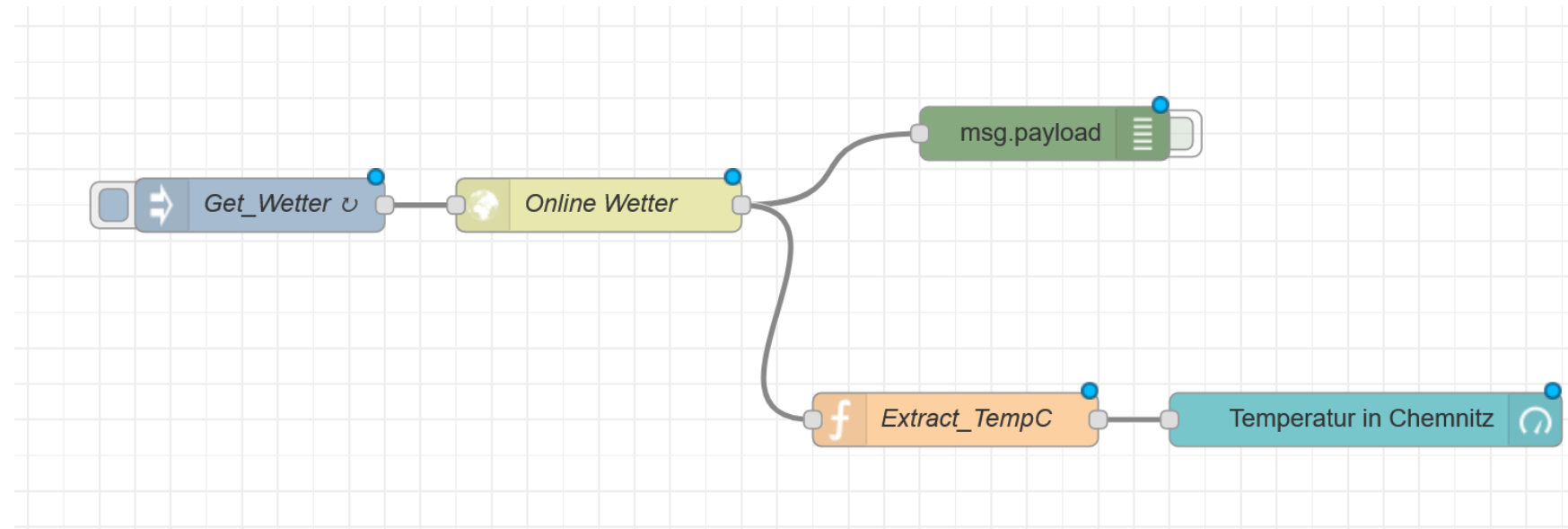
Node-Red

TTS: Text-to-speech



Alle Nachrichten an das Topic „UliFeder/Kanal“ werden durch den Lautsprecher als Sprache ausgegeben.
(solange man das NodeRed-Dashboard geöffnet hat)

Node-Red



Alle 10 Sekunden aktiviert die Node „Get_Wetter“ die nachfolgenden Node „Online Wetter“, welche auf die API von OpenWeatherMap zugreift. Wir erhalten dadurch eine große Menge an Daten aus der API

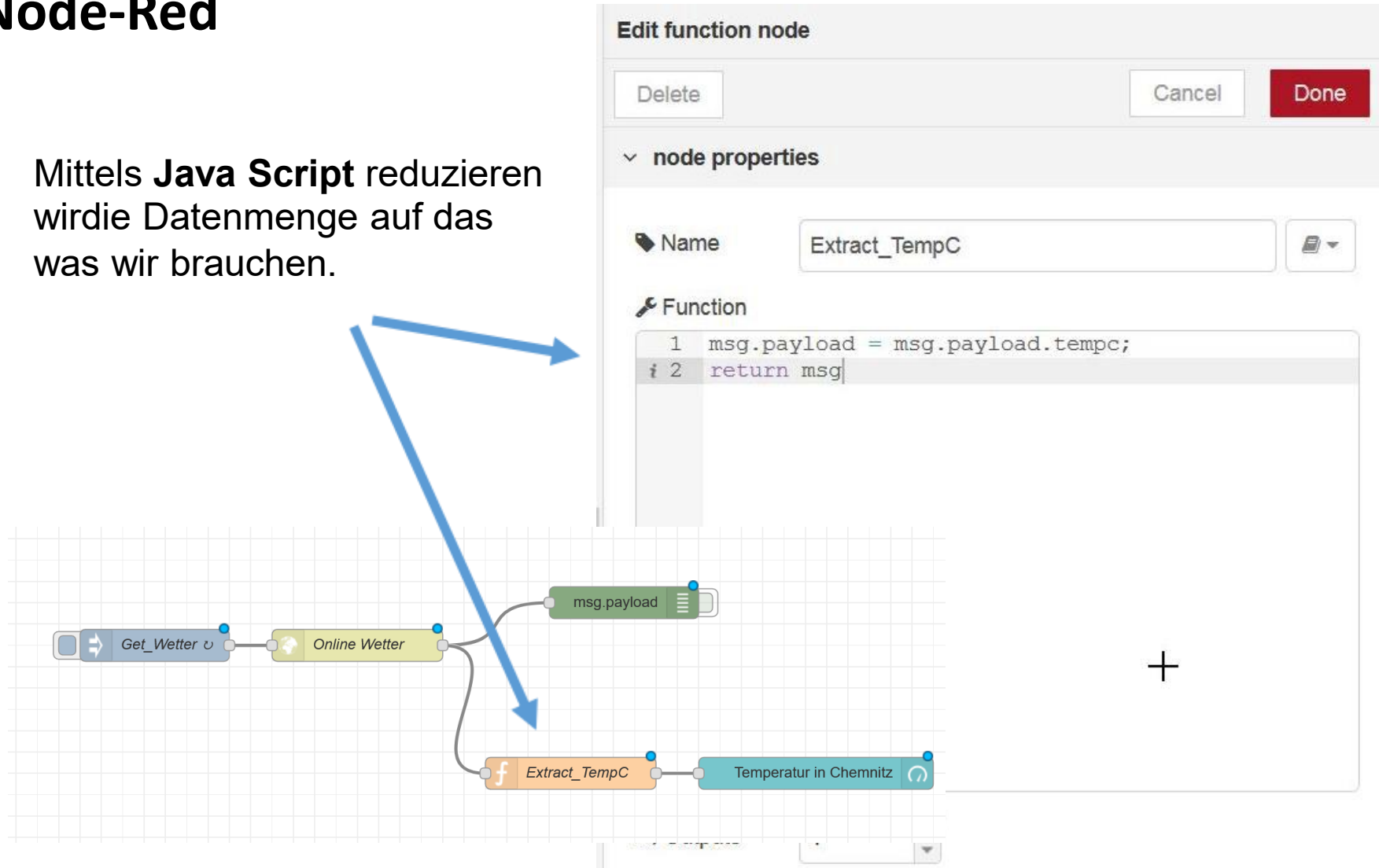
Wetterdaten:

<https://home.openweathermap.org>

(mein privater) API key: 98af392bd11624e1e496532423beac20

Node-Red

Mittels **Java Script** reduzieren
wirdie Datenmenge auf das
was wir brauchen.



Payload - msg

Was für Daten könnten wir alles bekommen?

Number

String

csv

Html

xml

wav

Video

JSON Objekt

→ häufig genutztes Format

The screenshot shows the Node-RED web interface. At the top, there are tabs for 'info', 'debug', and 'dashboard'. The 'debug' tab is selected. Below the tabs, there is a search bar with 'all nodes' and a trash icon. The main area displays a list of messages. The first message is selected, showing its details. The message is a JSON object with the following properties:

```
{  "weather": "Clouds",  "detail": "Überwiegend bewölkt",  "icon": "04n",  "tempk": 276.15,  "tempc": 3,  "temp_maxc": 3,  "temp_minc": 3,  "humidity": 93,  "maxtemp": 276.15,  "mintemp": 276.15,  "windspeed": 0.5,  "location": "Chemnitz",  "sunrise": 1515654399,  "sunset": 1515684405,  "clouds": 75,  "description": "The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt)."} 
```

The second message is partially visible below the first one, showing it is a number.

- **JSON – JavaScript Object Notation**

Ausgabe aus OpenWeatherMap:

```
{"weather":"Clouds","detail":"Überwiegend bewölkt","icon":"04n","tempk":276.15,"tempc":3,"temp_maxc":3,"temp_minc":3,"humidity":93,"maxtemp":276.15,"mintemp":276.15,"windspeed":0.5,"location":"Chemnitz","sunrise":1515654399,"sunset":1515684405,"clouds":75,"description":"The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt)."}}
```



• JSON – JavaScript Object Notation

```
{"weather":"Clouds",  
  "detail":"Überwiegend bewölkt",  
  "icon":"04n",  
  "tempk":276.15,  
  "tempc":3,  
  "temp_maxc":3,  
  "temp_minc":3,  
  "humidity":93,  
  "maxtemp":276.15,  
  "mintemp":276.15,  
  "windspeed":0.5,  
  "location":"Chemnitz",  
  "sunrise":1515654399,  
  "sunset":1515684405,  
  "clouds":75,  
  "description":"The weather in Chemnitz at  
coordinates: 50.83, 12.93 is Clouds (Überwiegend  
bewölkt)."}}
```



- **JSON – JavaScript Object Notation**

... is a lightweight data-interchange format.

... completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others.

A collection of name/value pairs (objects).

An ordered list of values (arrays).

(Zitate aus <https://www.json.org/>)

Beispiel „Kreditkarte“

(aus https://de.wikipedia.org/wiki/JavaScript_Object_Notation)

```
{
  "Herausgeber": "Xema",
  "Nummer": "1234-5678-9012-3456",
  "Deckung": 2e+6,
  "Waehrung": "EURO",
  "Inhaber":
  {
    "Name": "Mustermann",
    "Vorname": "Max",
    "maennlich": true,
    "Hobbys": ["Reiten", "Golfen", "Lesen"],
    "Alter": 42,
    "Kinder": [],
    "Partner": null
  }
}
```

OpenHAB: open Home Automation Bus

<http://www.openhab.org/>

Automatisierungssoftware für das „Smart Home“

Steuert (Licht, Heizung, Stereoanlage, Waschmaschine, Katze,...)

Bildet Strukturen ab

- Keller, Erdgeschoss, Gartenlaube,...

- Wohnzimmer, Küche, Bibliothek,...

- Heizung, Licht, Kochen, Unterhaltung,...

Speichert Zustände (Licht an, Temperatur im Wohnzimmer,...)

Regelt (am Wochenende eine Stunde vor Sonnenaufgang das Bad heizen und die Katze füttern)

Viele „Bindings“ für Geräte von unterschiedlichen Herstellern nach unterschiedlichen Normen (KNX, ENOCEAN, HomeMatic, ...)

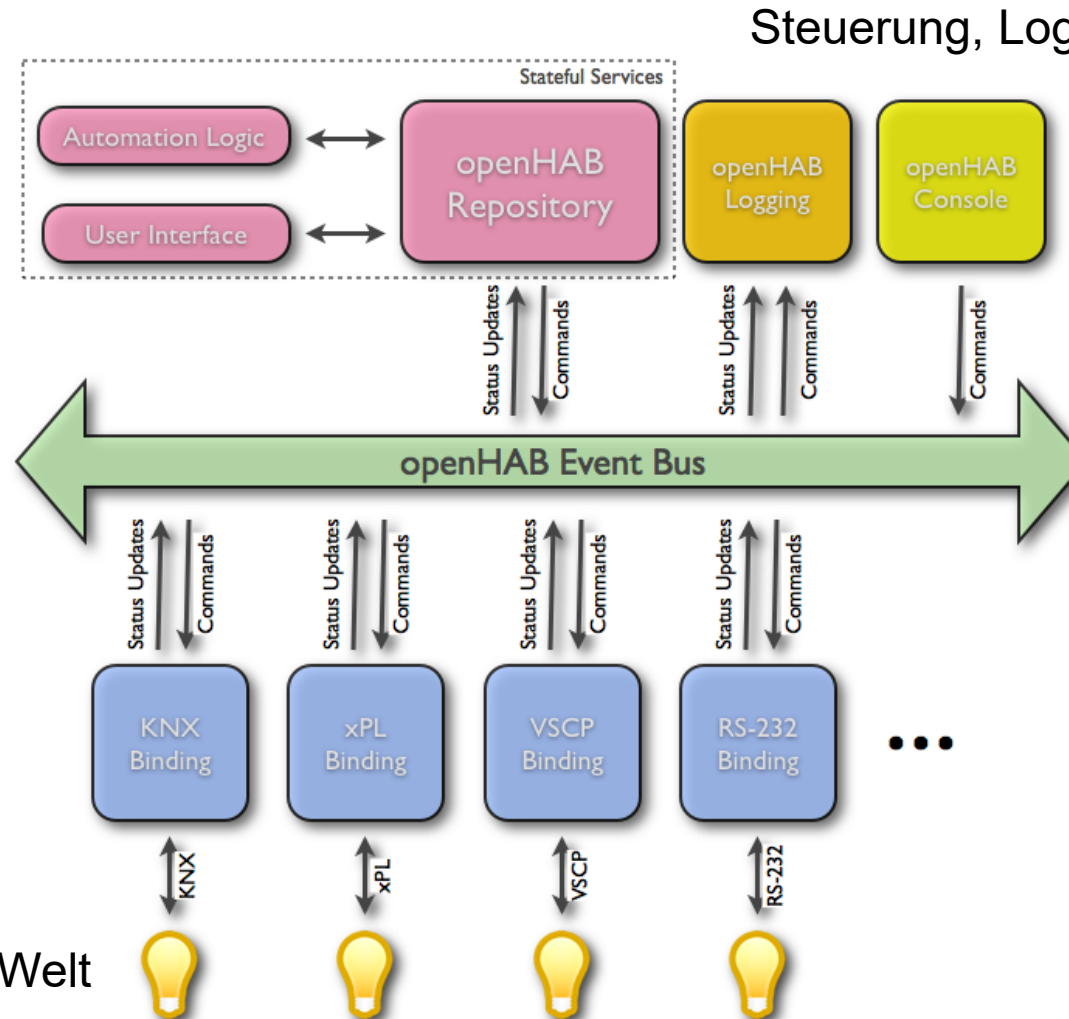
OpenHAB

Zustandsbeschreibung
und Automatisierung
(„Items“, „Rules“)

Asynchroner Ereignis-Bus

Bindings (nachlad-
und erweiterbar)

„Things“: Hardware in der echten Welt



Von Kai Kreuzer openHAB Entwickler - Eigenes [Werk](https://github.com/openhab/openhab/wiki/images/events.png)
Copyrighted free use, <https://commons.wikimedia.org/w/index.php?curid=32157828>

OpenHAB

OpenHAB gliedert sich in
Ausführender Server-Prozess: openHAB-runtime
„Benutzerfreundliche“ Konfigurationsoberfläche

Version 1.X und früher ist nur Server, die Konfiguration geht nur über Konfigurationsdateien, die „von Hand“ editiert werden müssen.

Version 2.X bietet eine Web-basierte Konfigurationsoberfläche an; trotzdem müssen Konfigurationsdateien noch editiert werden.

Die Konfiguration einer openHAB Lösung ist noch etwas für Experten, die Bedienung ist dann intuitiv über Web-Oberflächen.

OpenHAB Konfigurationsdateien

z.B. für Linux (Raspberry Pi) in den Verzeichnissen

/etc/openhab2/items

/etc/openhab2/sitemaps

/etc/openhab2/things

/etc/openhab2/rules

Text-Dateien mit den Endungen .items, .sitemap, .things, .rules

OpenHAB Konfigurationsdateien

Beispiel: Home.items Datei

Alle Objekte werden in den .items Dateien definiert (ähnlich einer Variablen- und Objekt-Deklaration)

```
Group Home          "Our Home"    <house>
Group Library       "Library"      <office>    (Home)
Group LivingRoom    "Living Room"  <sofa>      (Home)

Switch Library_Light "Light"       <light>      (Library, gLight)    {channel=""}
```

```
Number Library_Heating "Heating"    <heating>    (Library, gHeating)    {channel=""}
```

```
Number Library_Humidity "Humidity"  <humidity>    (Library, gHumidity)    {channel=""}
```

```
Number Library_Temperature "Temperature" <temperature> (Library, gTemperature) {channel=""}
```

```
Switch LivingRoom_Light "Light"       <light>      (LivingRoom, gLight)    {channel=""}
```

```
Number LivingRoom_Heating "Heating"    <heating>    (LivingRoom, gHeating)    {channel=""}
```

```
Number LivingRoom_Temperature "Temperature" <temperature> (LivingRoom, gTemperature) {channel=""}
```

```
Number LivingRoom_Humidity "Humidity"    <humidity>    (LivingRoom, gHumidity)    {channel=""}
```



```
Group:Switch:OR(ON, OFF) gLight "Light"    <light>    (Home)
```

```
Group:Number:AVG gHeating "Heating"    <heating>    (Home)
```

```
Group:Number:AVG gHumidity "Humidity"    <humidity>    (Home)
```

```
Group:Number:AVG gTemperature "Temperature" <temperature> (Home)
```

OpenHAB Konfigurationsdateien

Lichtschalter als Item:

```
Switch Library_Light      "Light"      <light>      (Library, gLight)      {channel=""}
```

Objekt ist ein Schalter (Switch) mit dem Namen („Variablennamen“) Library_Light und dem Label „Light“ (im Web-Interface sichtbare Bezeichnung), dem Icon <light>.

Das Objekt gehört zu den Gruppen (Library, gLight).

Mit dem Objekt ist die in {} eingetragene Aktion verknüpft (in diesem Fall nichts).

OpenHAB Konfigurationsdateien

Beispiel: Home.sitemap Datei beschreibt die Struktur.

```
sitemap our_home label="Our Home" {  
    Frame {  
        Group item=Library  
        Group item=LivingRoom  
    }  
    Frame {  
        Text label="Light" icon="light" {  
            Default item=Library_Light label="Library"  
            Default item=LivingRoom_Light label="Living Room"  
        }  
        Text label="Heating" icon="heating" {  
            Default item=Library_Heating label="Library"  
            Default item=LivingRoom_Heating label="Living Room"  
        }  
        Text label="Humidity" icon="humidity" {  
            Default item=Library_Humidity label="Library"  
            Default item=LivingRoom_Humidity label="Living Room"  
        }  
        Text label="Temperature" icon="temperature" {  
            Default item=Library_Temperature label="Library"  
            Default item=LivingRoom_Temperature label="Living Room"  
        }  
    }  
}
```

OpenHAB Konfigurationsdateien

Beispiel: Feder.items

Hier wird ein MQTT Binding eingesetzt, um mit das EPS8266 Board mit seinen Sensoren einzubinden.

```
Group      My      "EPS8266"      <office>

Switch     SchalterRoteLED      "RoteLED" <light> (Feder)
{ mqtt=">[mosquitto:LEDonBoard:command:ON:true],>[mosquitto:LEDonBoard:command:OFF:false]" }

Number     My_Humidity      "Humidity [%].2f rel. percent]" <humidity> (Feder)
{ mqtt="<[mosquitto:Sensors/Humidity:state:default]" }

Number     My_Temperature      "Temperature [%].1f °C]" <temperature> (Feder)
{ mqtt="<[mosquitto:Sensors/Temperature:state:default]" }

Number     My_Pressure      "Luftdruck [%].0f Pa]" <pressure> (Feder)
{ mqtt="<[mosquitto:Sensors/Pressure:state:default]" }
```

OpenHAB Rules

Dateien `RegelName_oder_so.rules` in `/etc/openhab2/rules`

Deklaration von Variablen (gelten innerhalb einer Regel-Datei):

```
var counter = 0           // a variable with an initial value. Note that the variable type is automatically inferred
val msg = "This is a message" // a read-only value, again the type is automatically inferred
```

Regeln:

```
rule "<RULE_NAME>"
when
    <TRIGGER_CONDITION> [or <TRIGGER_CONDITION2> [or ...]]
then
    <SCRIPT_BLOCK>
end
```

OpenHAB Rules

Ereignisse können Regeln auslösen (triggern):

Item <item> received command [<command>]

Item <item> received update [<state>]

Item <item> changed [from <state>] [to <state>]

Von Timer ausgelöste Regeln:

Time is midnight

Time is noon

Time cron "<cron expression>"

... und weitere Möglichkeiten

OpenHAB Rules

Beispiel Dimmer:

```
rule "Dimmen Light"
```

```
when
```

```
    ITEM DimmndLight received command
```

```
then
```

```
    var Number percent = 0
```

```
    if(DimmndLight.state instanceof DecimalType) percent = DimmndLight.state as DecimalType
```

```
    if(receivedCommand == INCREASE) percent = percent + 5
```

```
    if(receivedCommand == DECREASE) percent = percent - 5
```

```
    if(percent < 0) percent = 0
```

```
    if(percent > 100) percent = 100
```

```
    postUpdate(DimmndLight, percent);
```

```
End
```

OpenHAB Rules

Beispiel: „Channel“ des „Astro“ Binding löst eine Regeln aus:

```
rule "Start wake up light on sunrise"
when
    Channel "astro:sun:home:rise#event" triggered START
then
    ...
End
```

Online Documents

MQTT

<https://www.heise.de/developer/artikel/MQTT-Protokoll-fuer-das-Internet-der-Dinge-2168152.html>

Node-Red

<https://nodered.org/>

JSON

<https://www.json.org/>

<https://en.wikipedia.org/wiki/JSON>

https://de.wikipedia.org/wiki/JavaScript_Object_Notation

openHAB

<https://docs.openhab.org/>

<https://docs.openhab.org/concepts/index.html>

<https://docs.openhab.org/tutorials/beginner/index.html>

Literaturempfehlung für den tieferen Einstieg: Walter Trojan, Das MQTT-Praxisbuch – mit ESP8266 und Node-RED (elektor Verlag, 2017, 1. Auflage)